Practical 4 ML Lab

Name : Ishan Manekar

Class : A_A3_45

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df = pd.read_csv("winequality-red.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

In [4]:
```
df.tail()
```

Out[4]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
1591	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5
1592	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6
1593	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6
1594	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5
1595	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0	6

In [5]:
```
df.shape
```

Out[5]:
```
(1596, 12)
```

In [6]:
```
df.info
```

Out[6]:
```
<bound method DataFrame.info of       fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0               7.4             0.700         0.00             1.9      0.076
1               7.8             0.880         0.00             2.6      0.098
2               7.8             0.760         0.04             2.3      0.092
3              11.2             0.280         0.56             1.9      0.075
4               7.4             0.700         0.00             1.9      0.076
...             ...               ...          ...             ...        ...
1591            6.2             0.600         0.08             2.0      0.090
1592            5.9             0.550         0.10             2.2      0.062
1593            6.3             0.510         0.13             2.3      0.076
1594            5.9             0.645         0.12             2.0      0.075
1595            6.0             0.310         0.47             3.6      0.067

      free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                    11.0                  34.0  0.99780  3.51       0.56
1                    25.0                  67.0  0.99680  3.20       0.68
2                    15.0                  54.0  0.99700  3.26       0.65
3                    17.0                  60.0  0.99800  3.16       0.58
4                    11.0                  34.0  0.99780  3.51       0.56
...                   ...                   ...      ...   ...        ...
1591                 32.0                  44.0  0.99490  3.45       0.58
1592                 39.0                  51.0  0.99512  3.52       0.76
1593                 29.0                  40.0  0.99574  3.42       0.75
1594                 32.0                  44.0  0.99547  3.57       0.71
1595                 18.0                  42.0  0.99549  3.39       0.66

      alcohol  quality
0         9.4        5
1         9.8        5
2         9.8        5
3         9.8        6
4         9.4        5
...       ...      ...
1591     10.5        5
1592     11.2        6
1593     11.0        6
1594     10.2        5
1595     11.0        6

[1596 rows x 12 columns]>
```

In [7]:
```
df.describe()
```

Out[7]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	al
count	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.000000	1596.0(
mean	8.314160	0.527954	0.270276	2.535558	0.087120	15.858396	46.382206	0.996744	3.311917	0.656385	10.4
std	1.732203	0.179176	0.193894	1.405515	0.045251	10.460554	32.839138	0.001888	0.153346	0.163057	1.0
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.860000	0.330000	8.4(
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000000	0.995600	3.210000	0.550000	9.5(
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000	0.996745	3.310000	0.620000	10.2(
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000000	0.997833	3.400000	0.730000	11.1(
max	15.600000	1.580000	0.790000	15.500000	0.611000	72.000000	289.000000	1.003690	4.010000	1.980000	14.0(

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

In [9]:
```
df.columns
```

Out[9]:
```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'quality'],
      dtype='object')
```

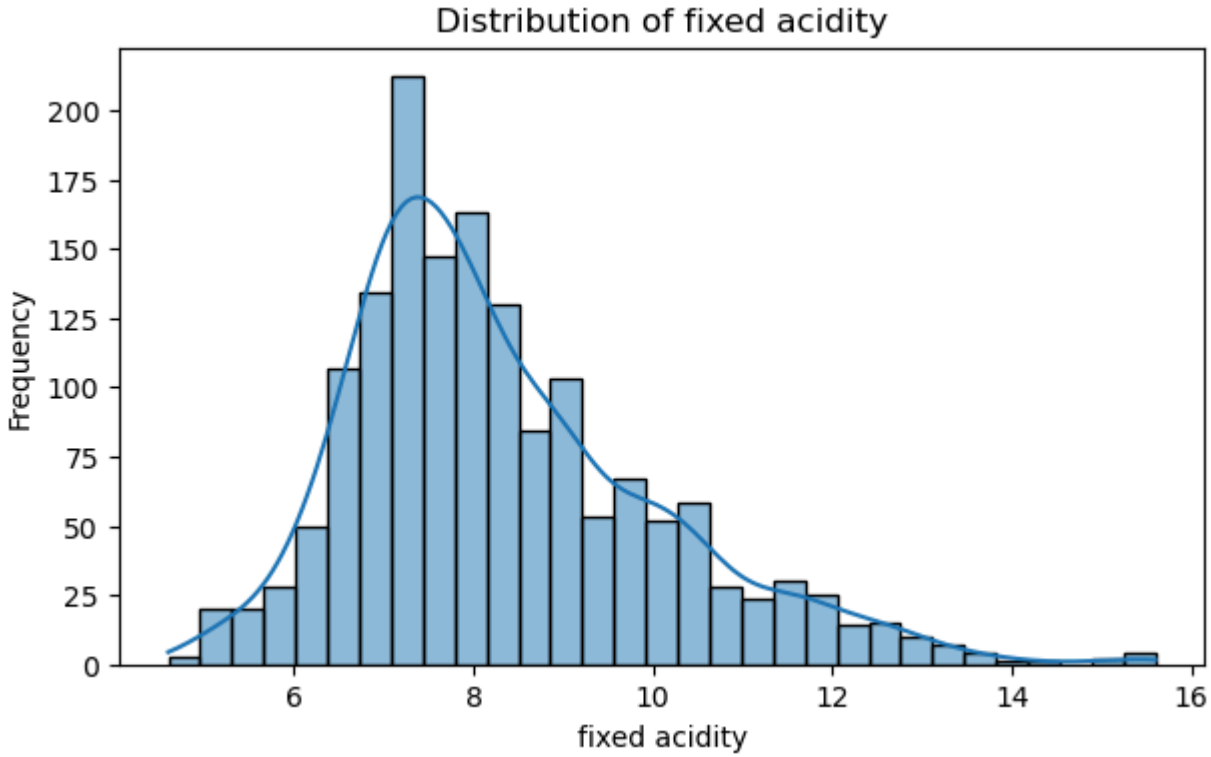
In [10]:
```
df.duplicated().sum()
```

Out[10]:
```
239
```

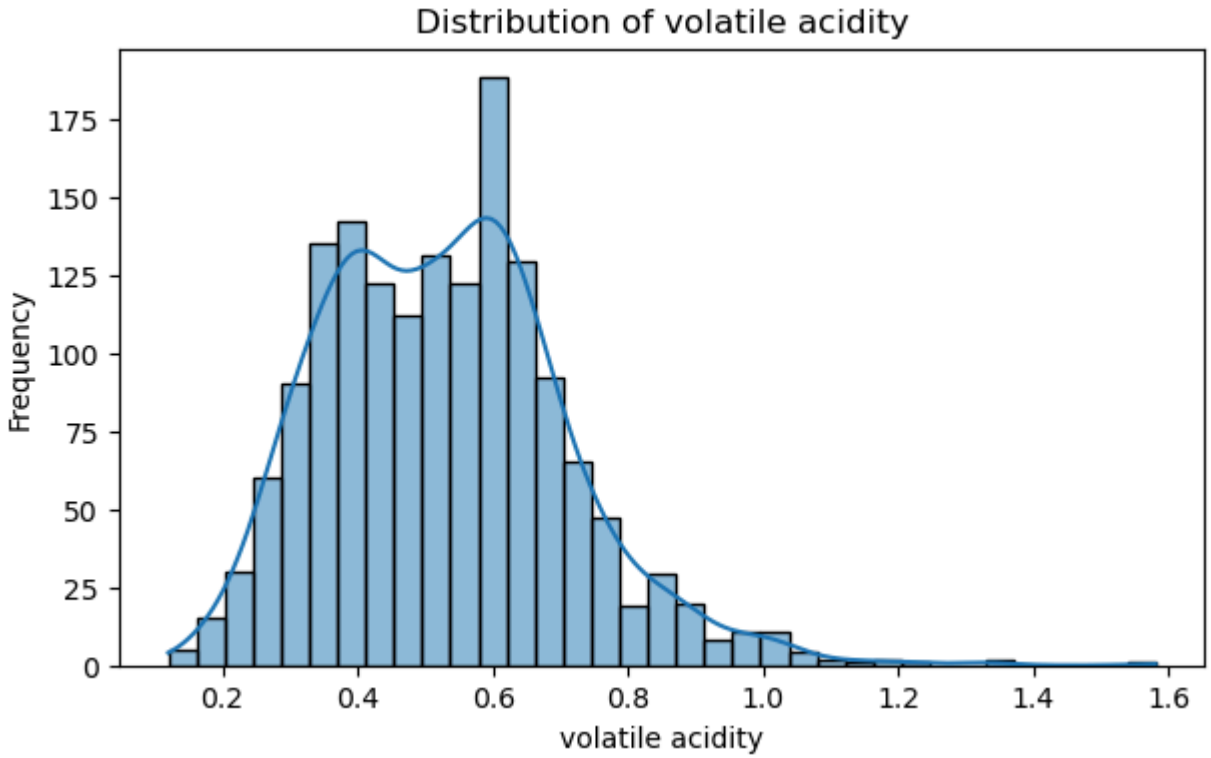
Univariate Analysis

In [11]:
```
#histogram +kde
numeric_cols = df.columns
for col in numeric_cols:
    plt.figure(figsize=(7,4))
    sns.histplot(df[col],kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.show()
```

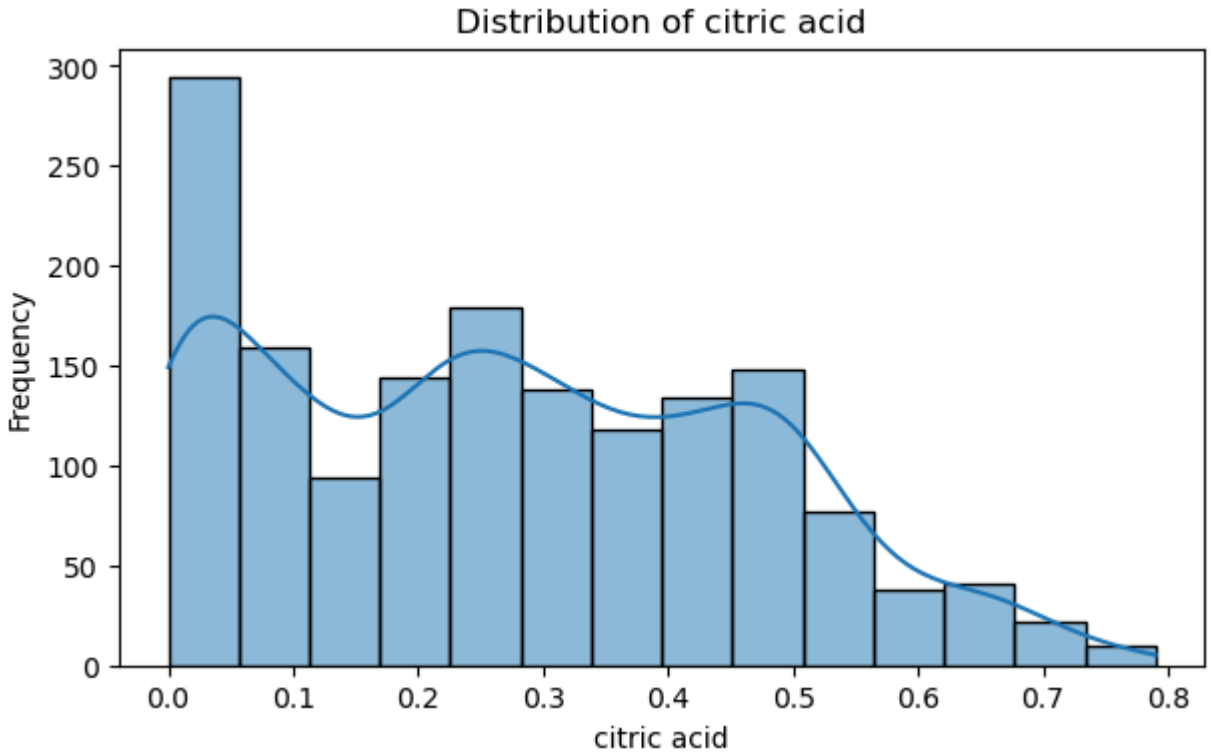
```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



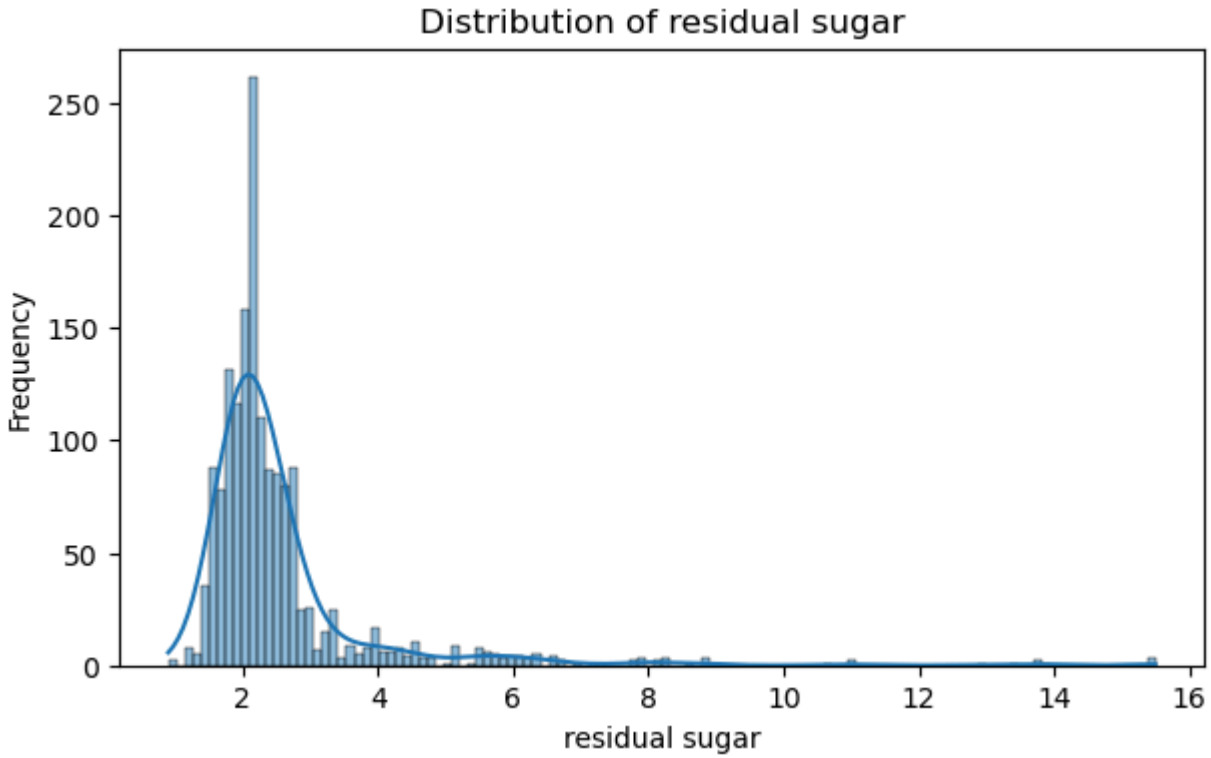

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



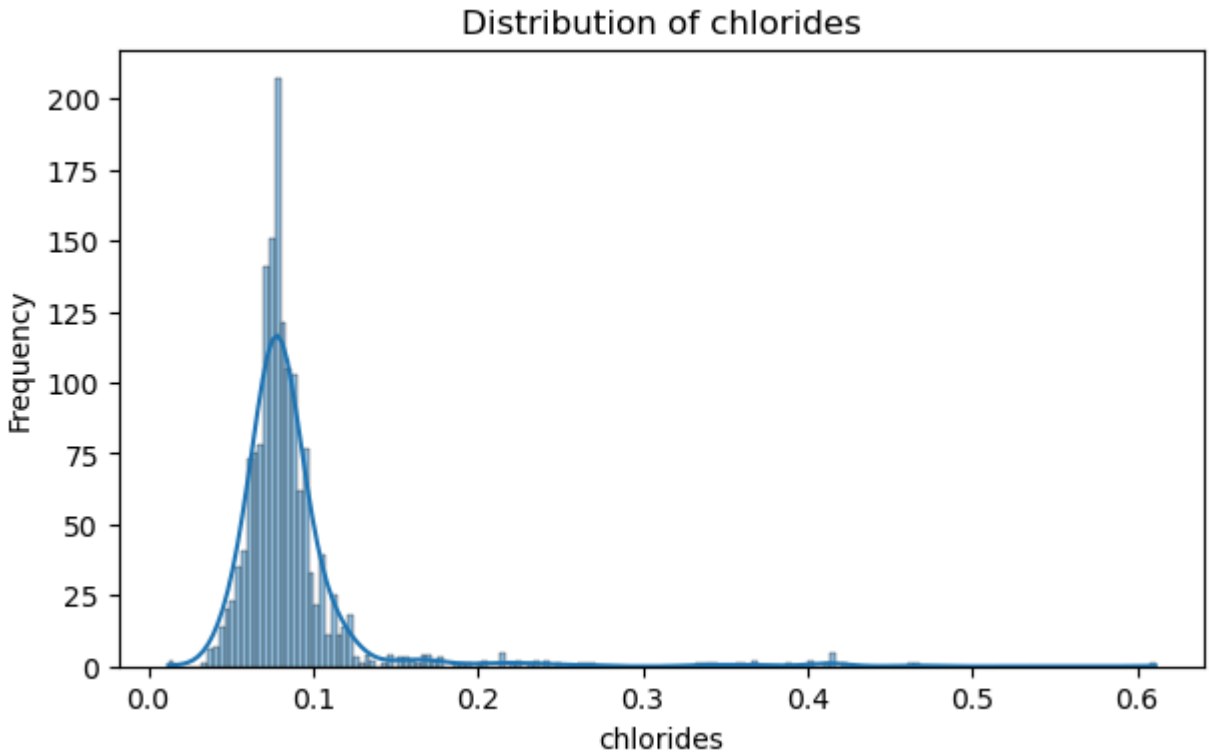

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



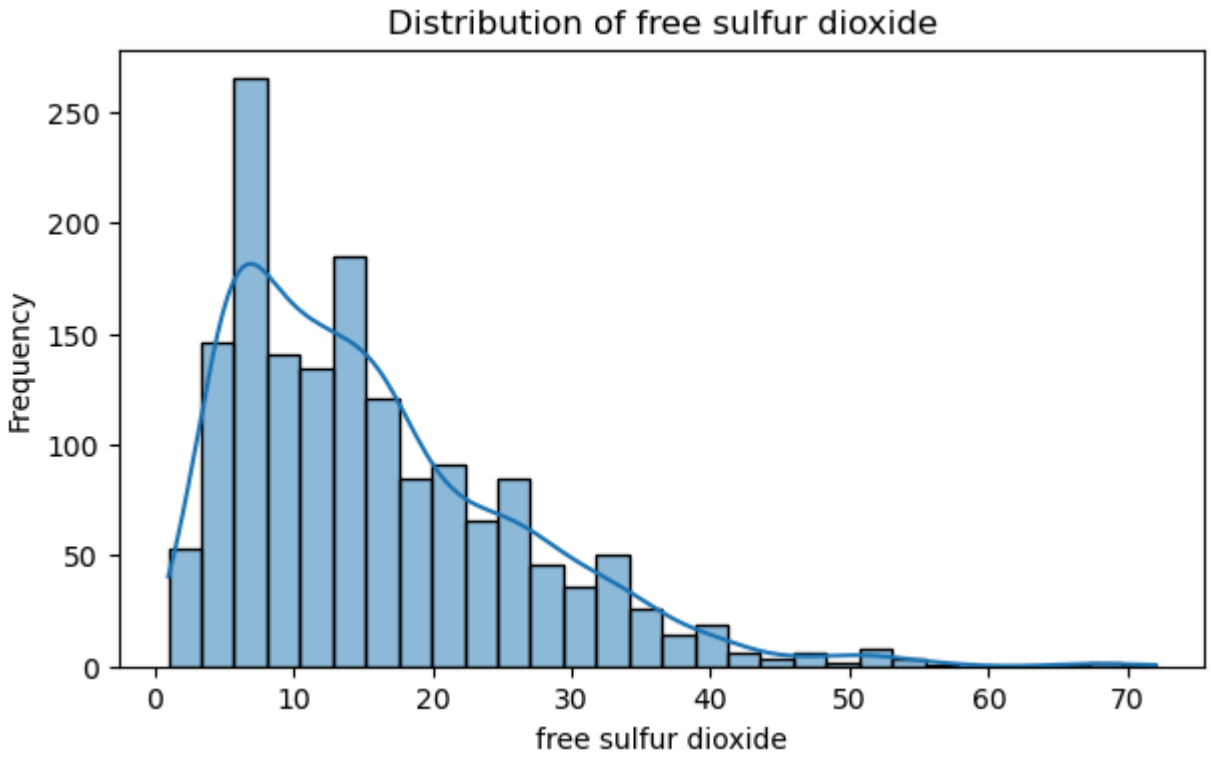

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



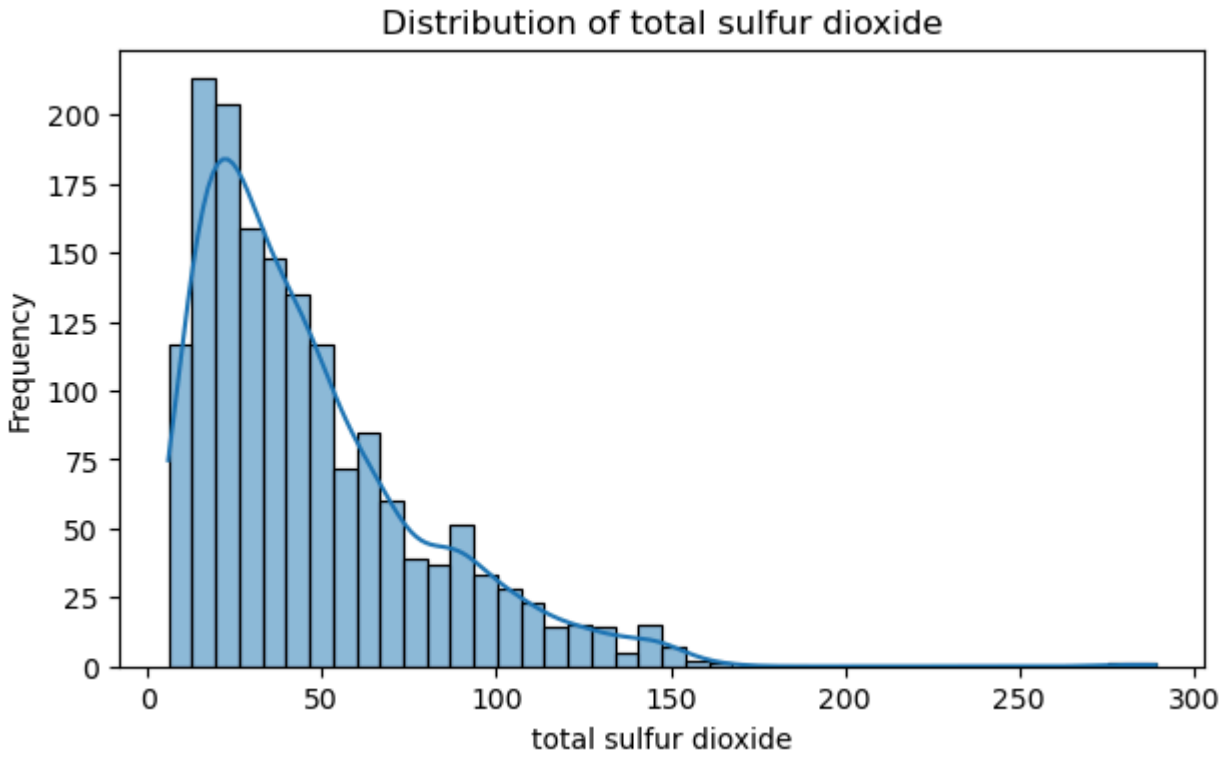

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



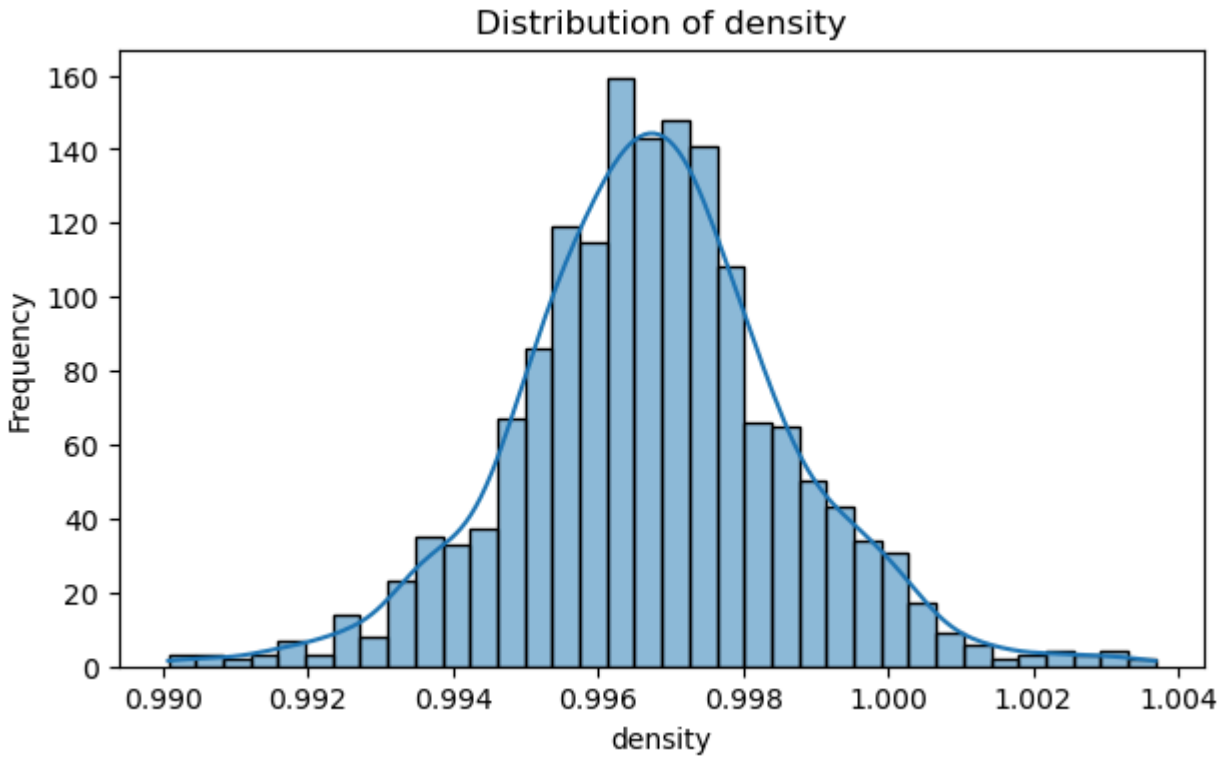

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



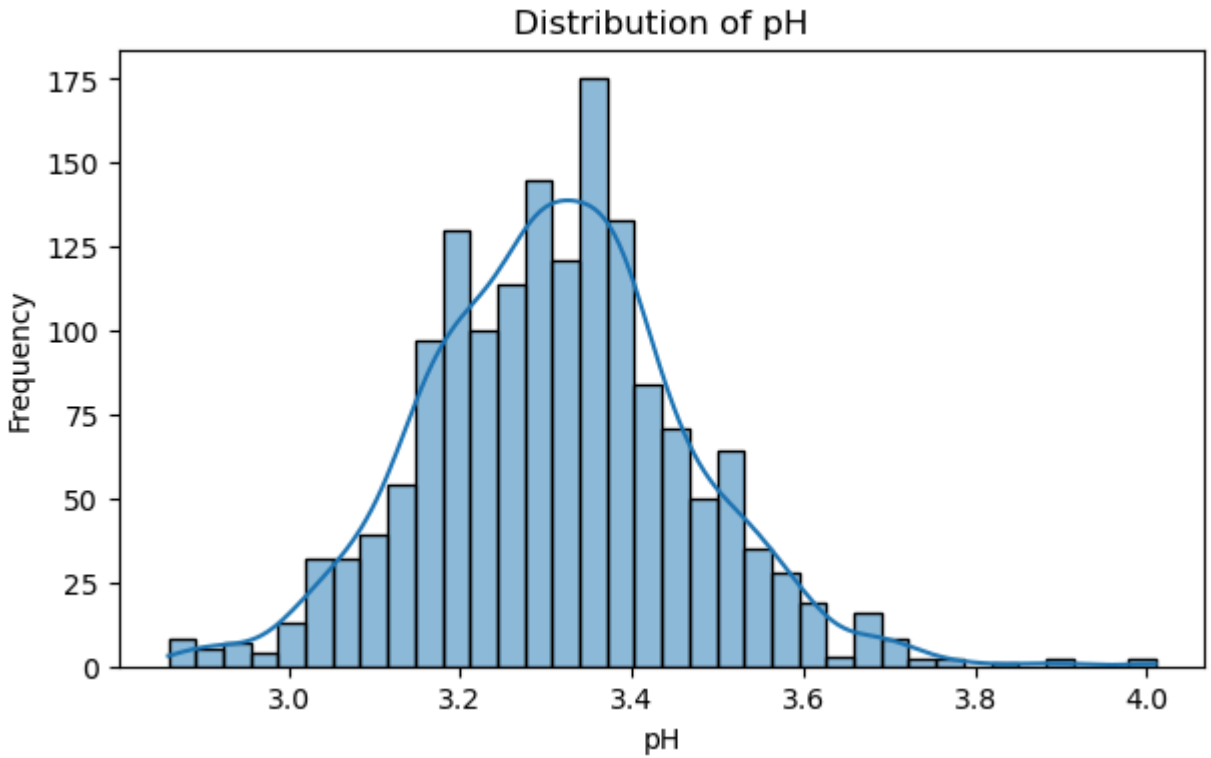

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



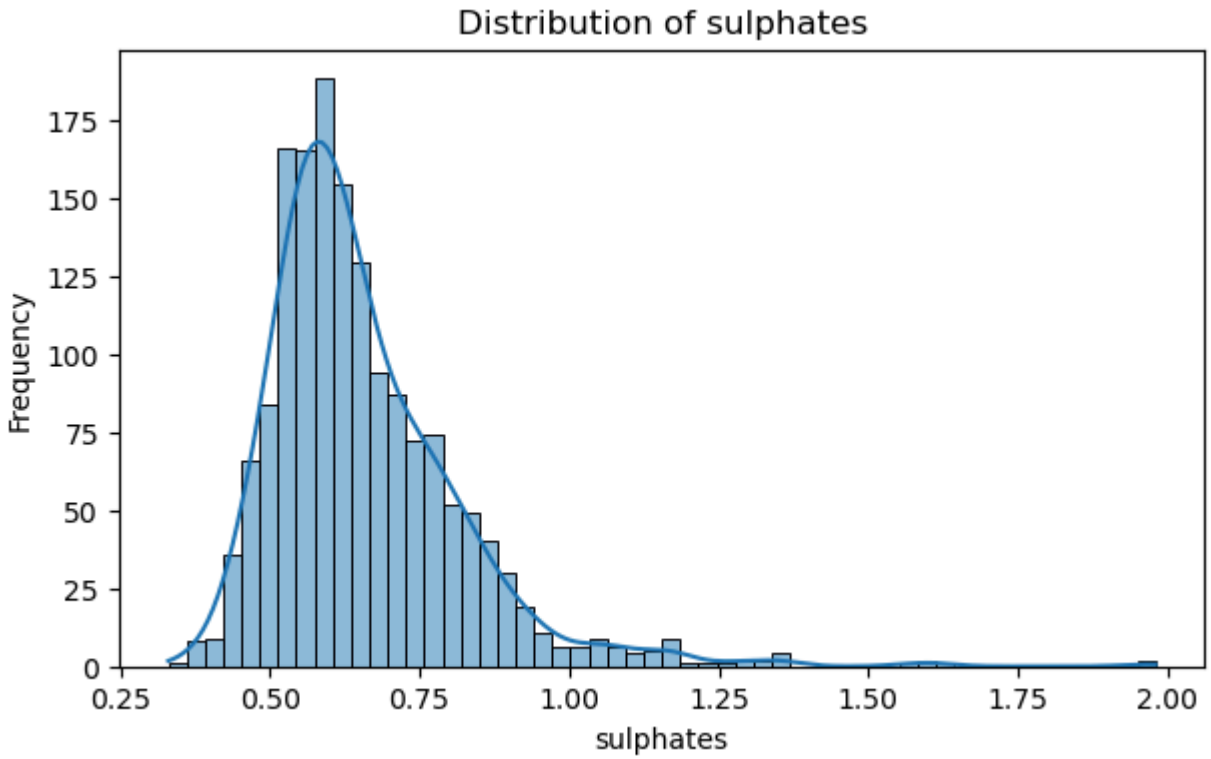

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



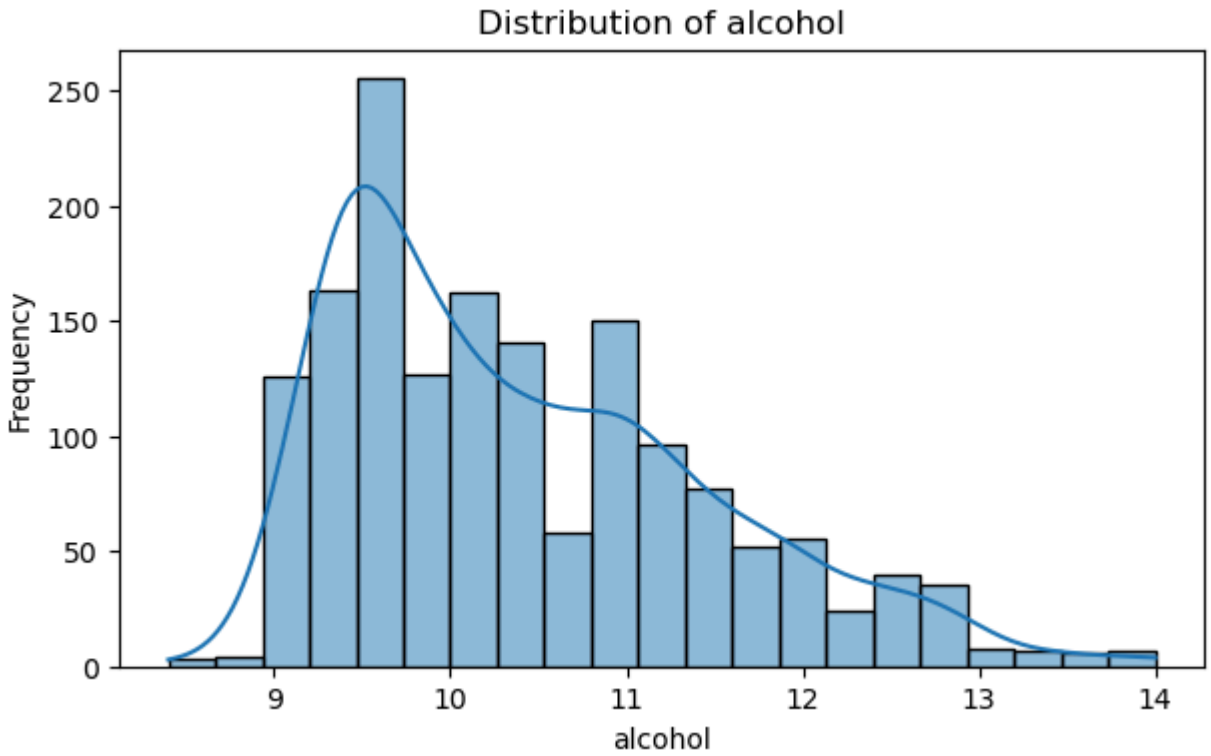

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



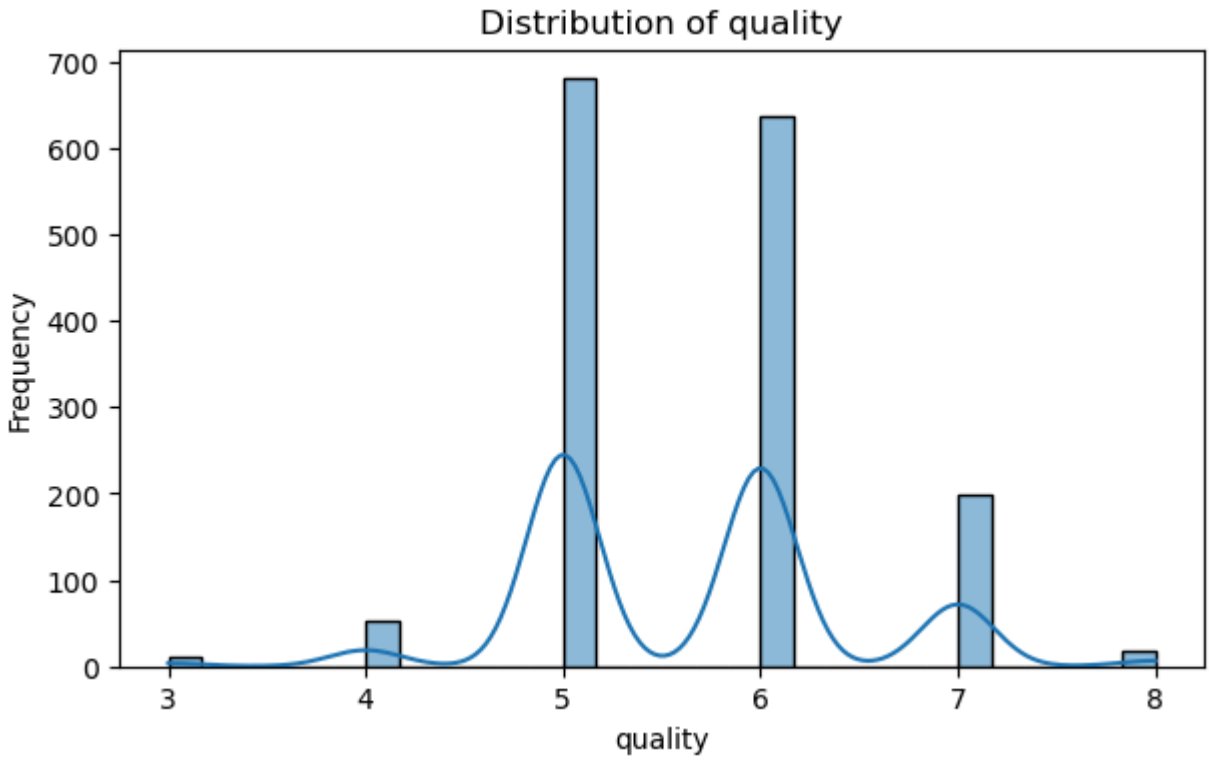

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```



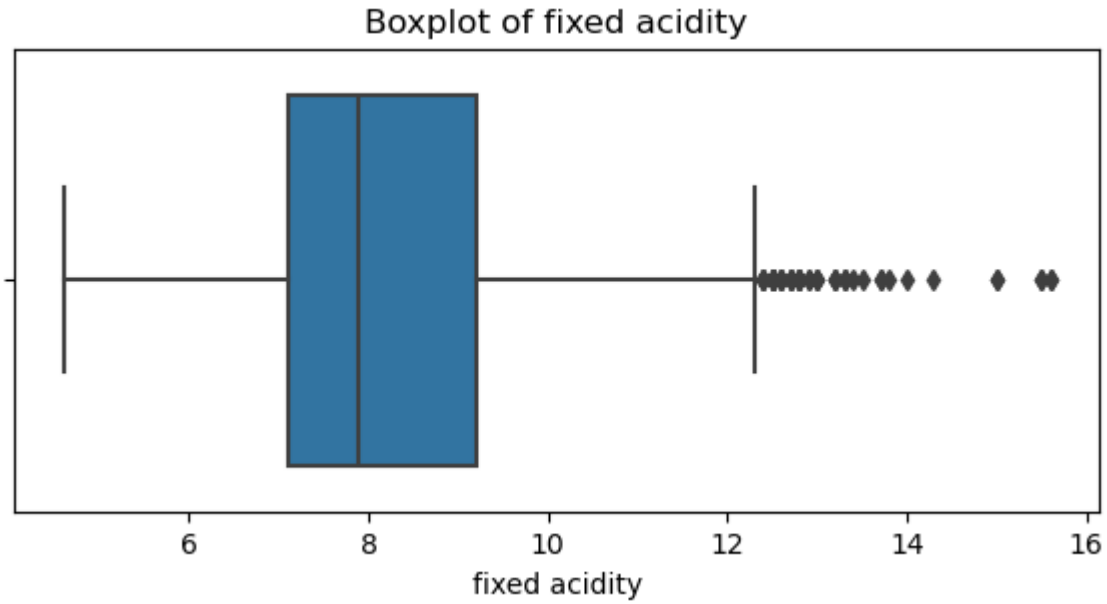

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

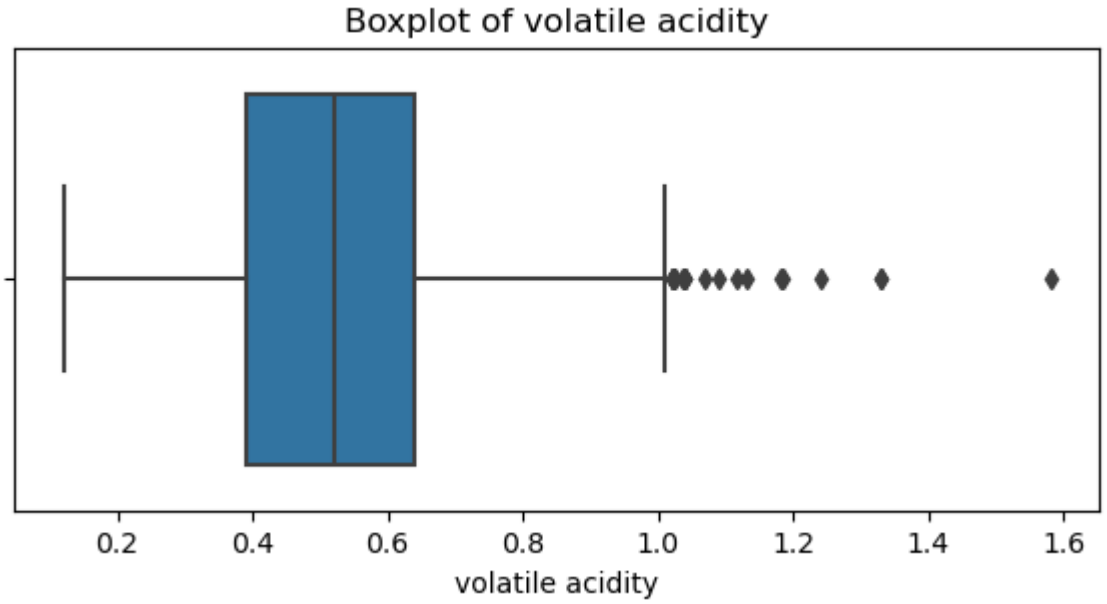


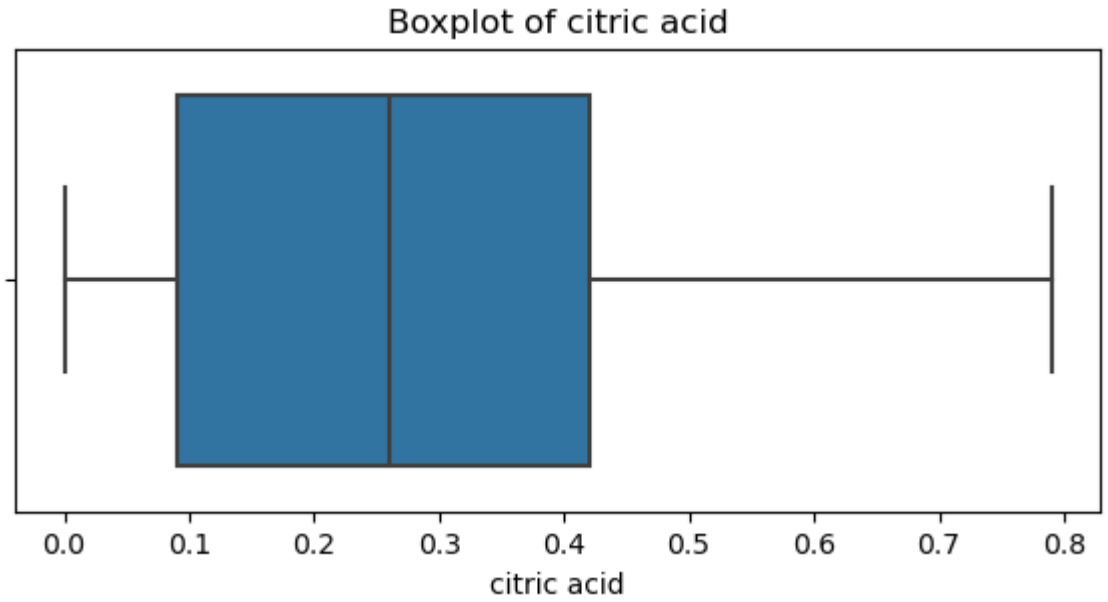

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

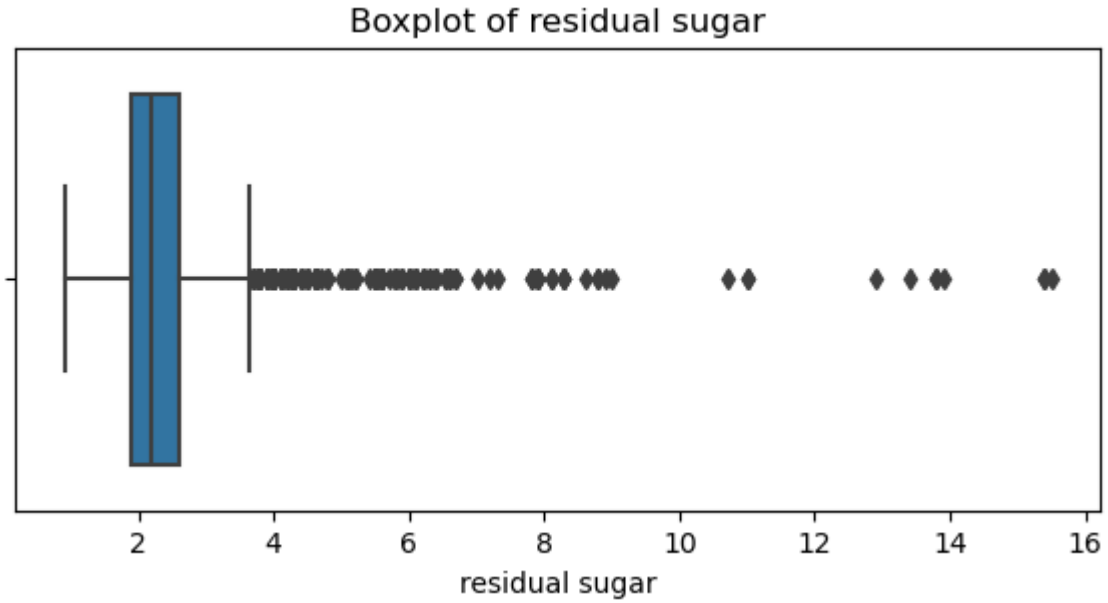

Distribution of quality
Frequency
quality
0
100
200
300
400
500
600
700
3
4
5
6
7
8

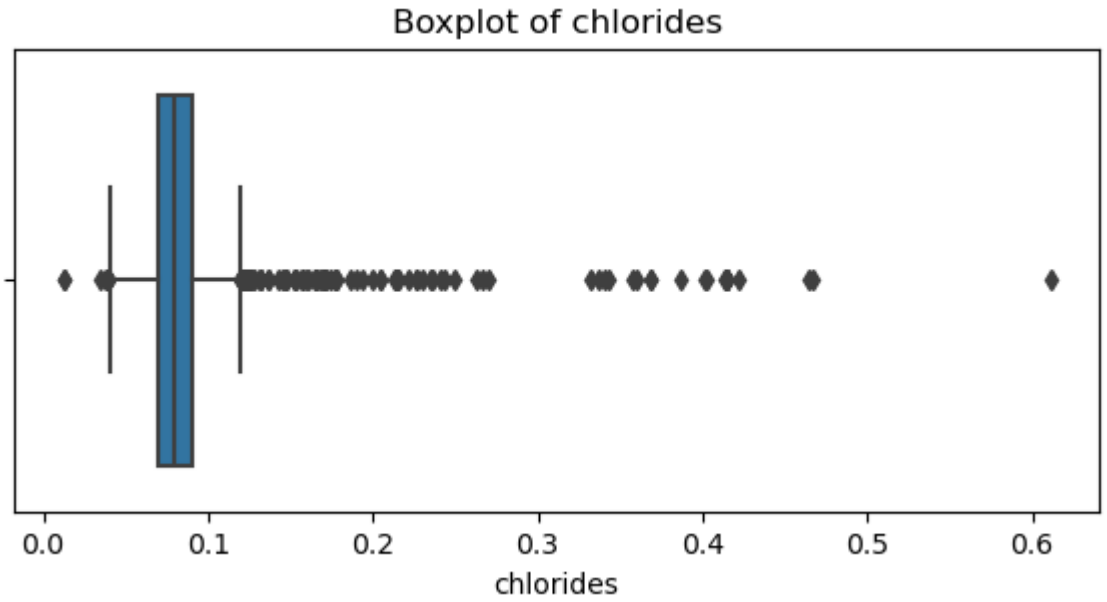
In [12]:
```
#Boxplot
for col in numeric_cols:
    plt.figure(figsize=(7,3))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()
```

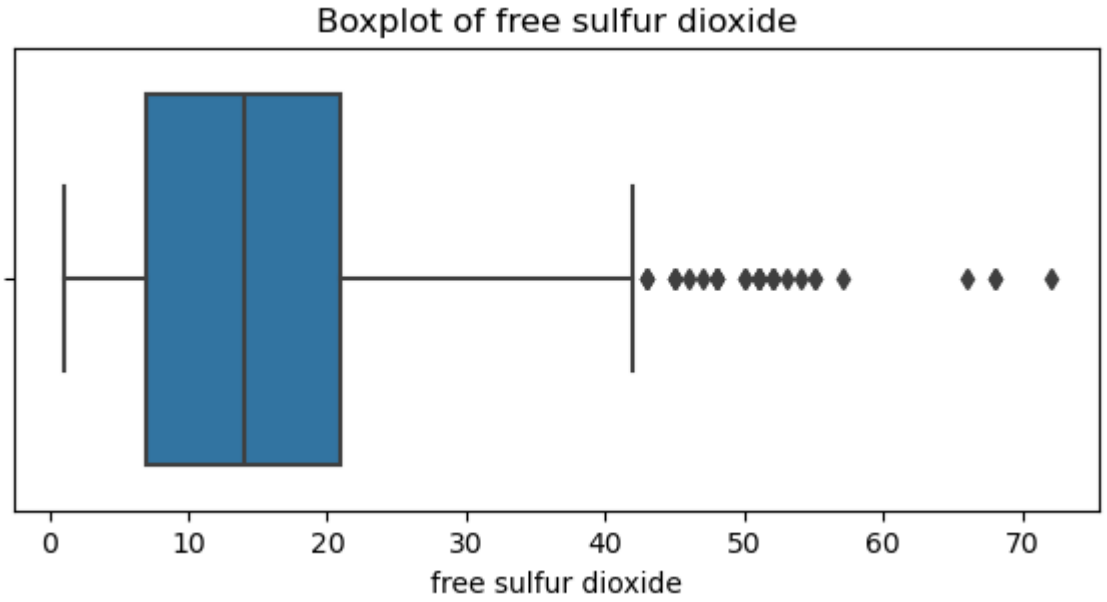

Boxplot of fixed acidity
6
8
10
12
14
16
fixed acidity

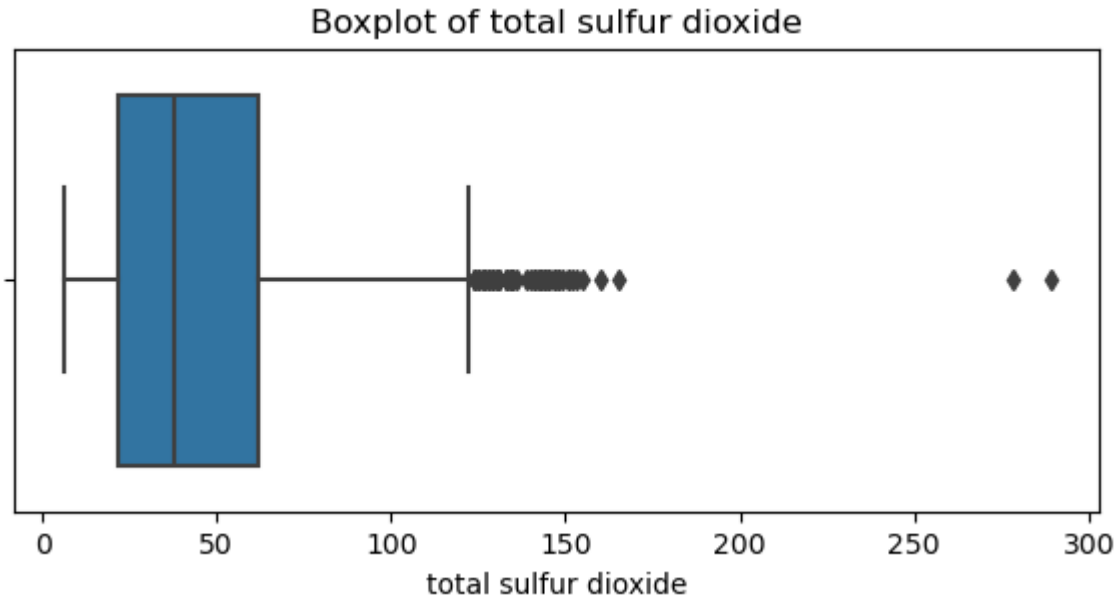

Boxplot of volatile acidity
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
volatile acidity

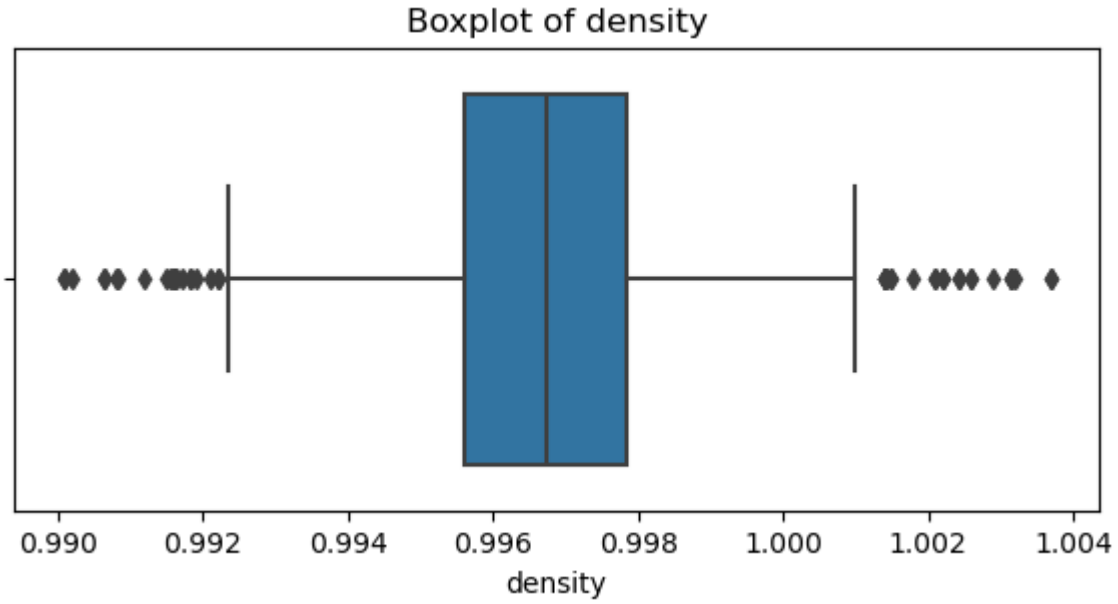

Boxplot of citric acid
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
citric acid

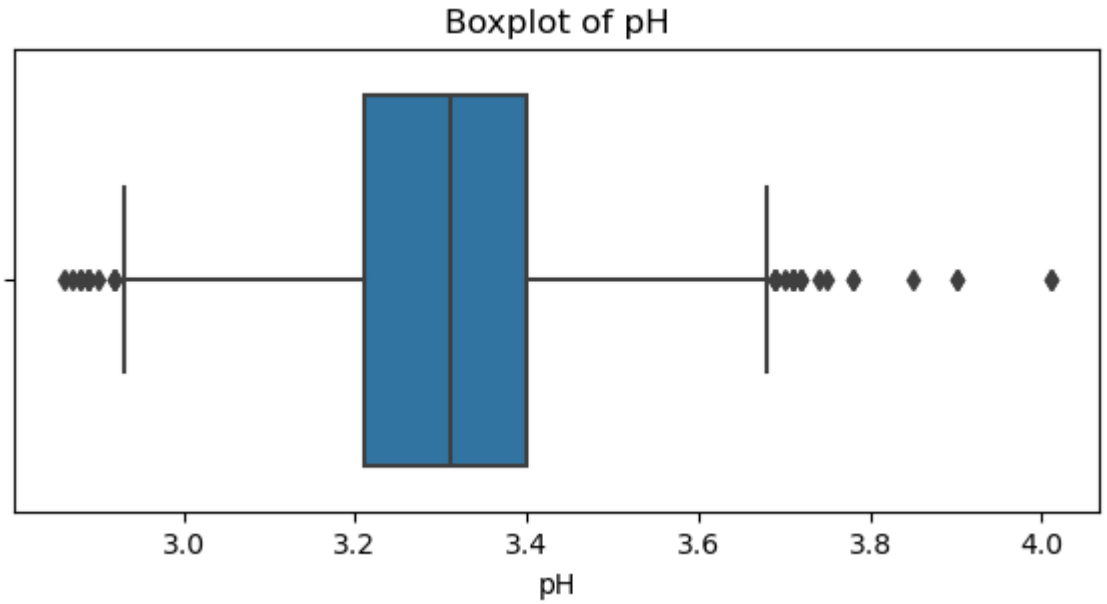

Boxplot of residual sugar
2
4
6
8
10
12
14
16
residual sugar

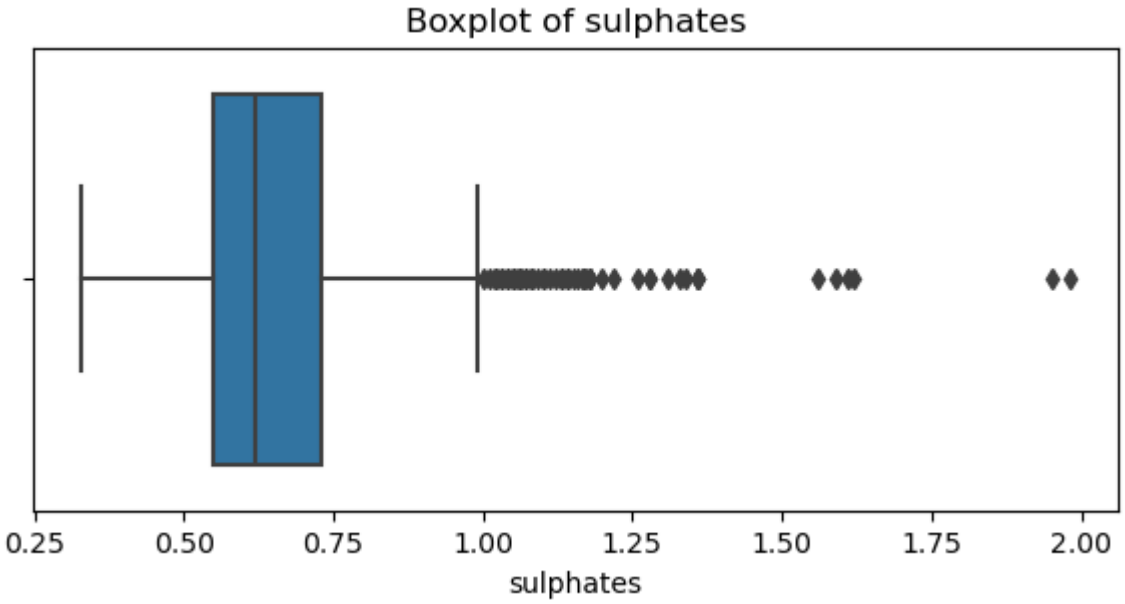

Boxplot of chlorides
0.0
0.1
0.2
0.3
0.4
0.5
0.6
chlorides

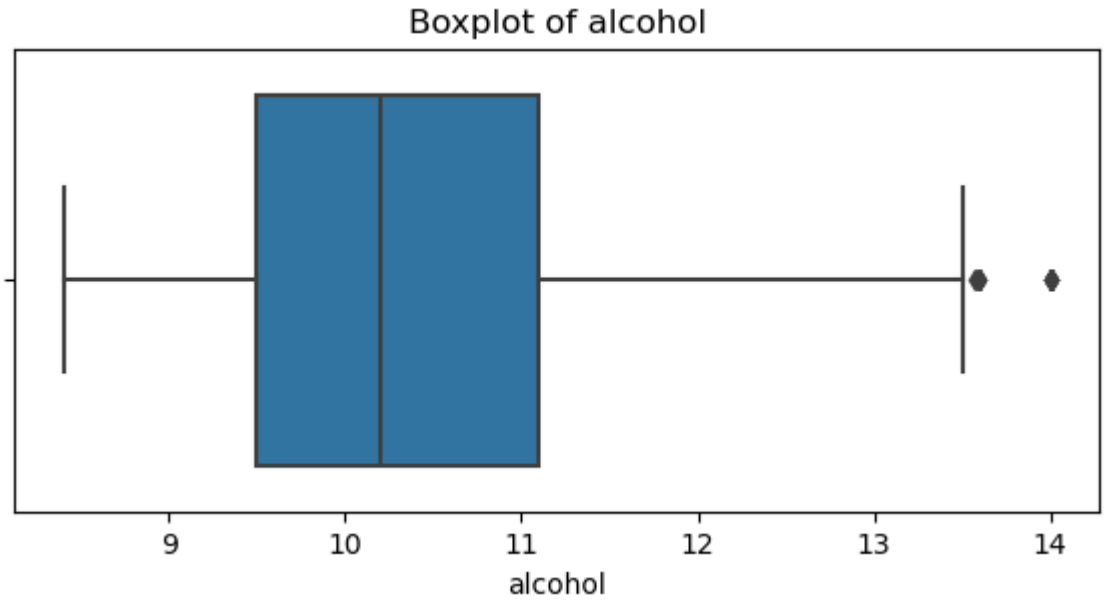

Boxplot of free sulfur dioxide
0
10
20
30
40
50
60
70
free sulfur dioxide

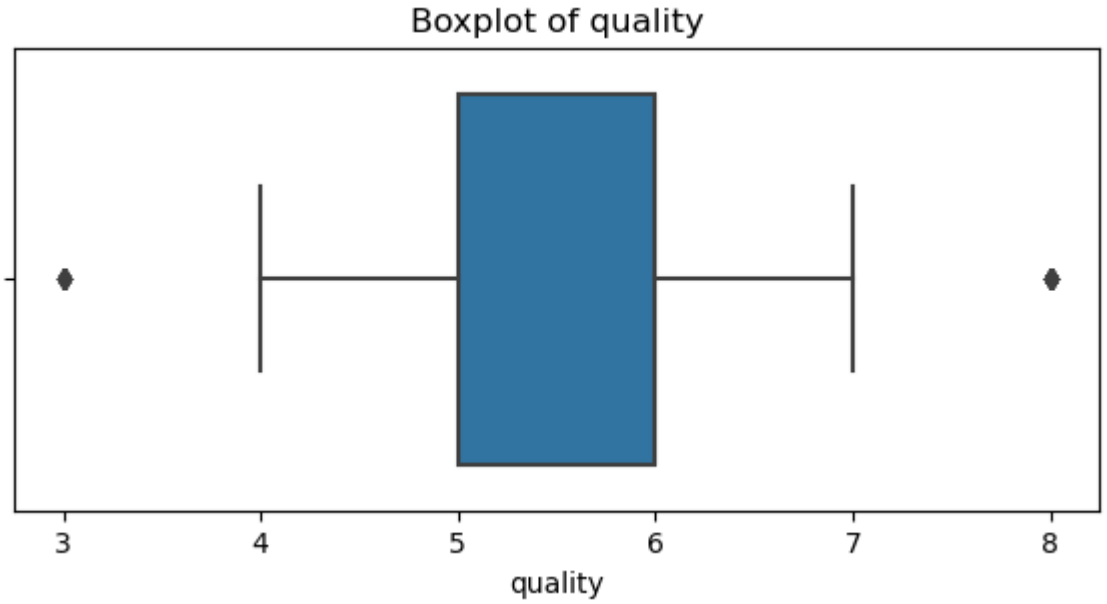

Boxplot of total sulfur dioxide
0
50
100
150
200
250
300
total sulfur dioxide


Boxplot of density
0.990
0.992
0.994
0.996
0.998
1.000
1.002
1.004
density

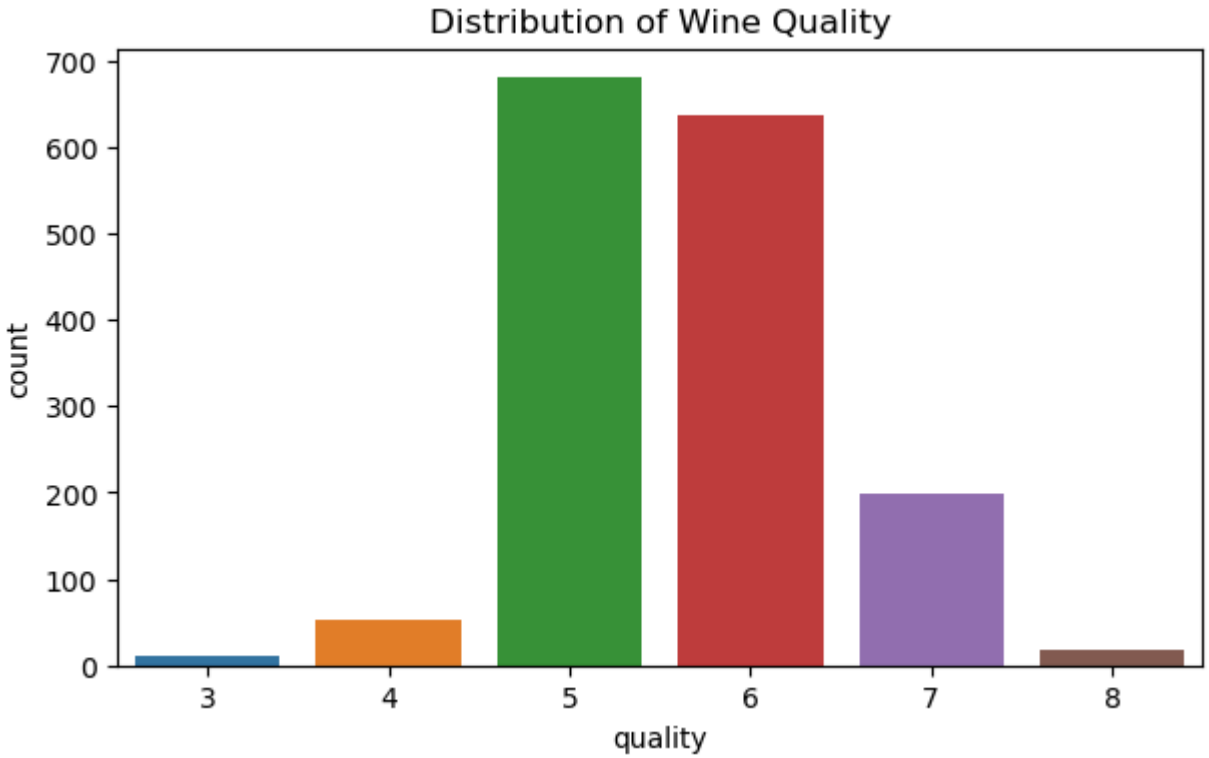

Boxplot of pH
3.0
3.2
3.4
3.6
3.8
4.0
pH


Boxplot of sulphates
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
sulphates


Boxplot of alcohol
9
10
11
12
13
14
alcohol


Boxplot of quality
3
4
5
6
7
8
quality

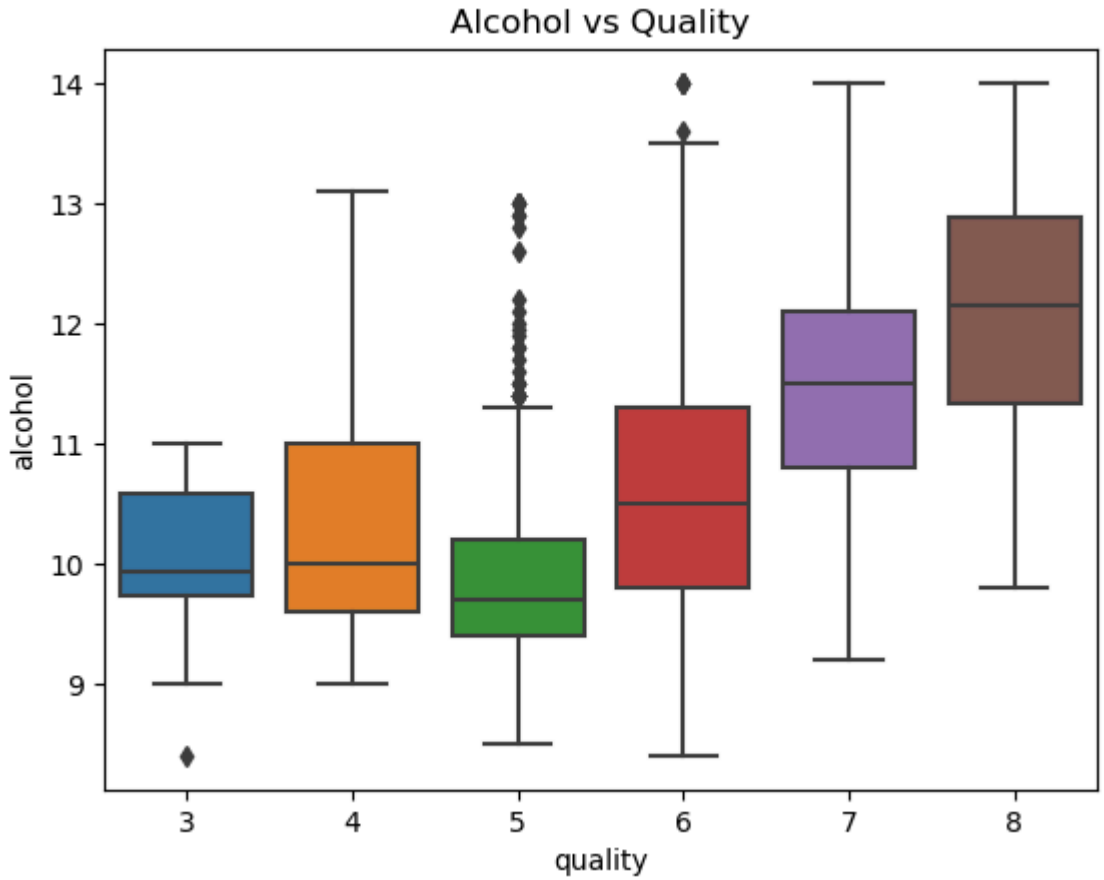
In [13]:
```
# quality sitribution
plt.figure(figsize=(7,4))
sns.countplot(x='quality',data=df)
plt.title("Distribution of Wine Quality")
plt.show()
```



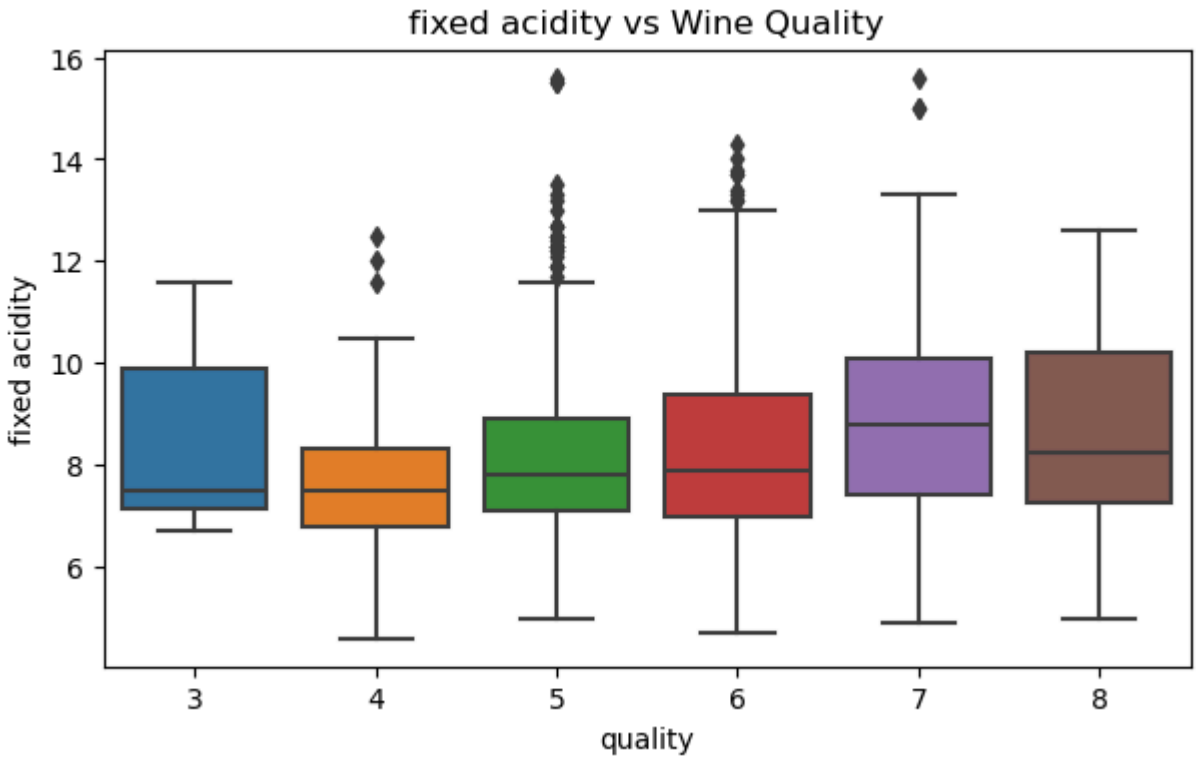
2 Bivariate Analysis

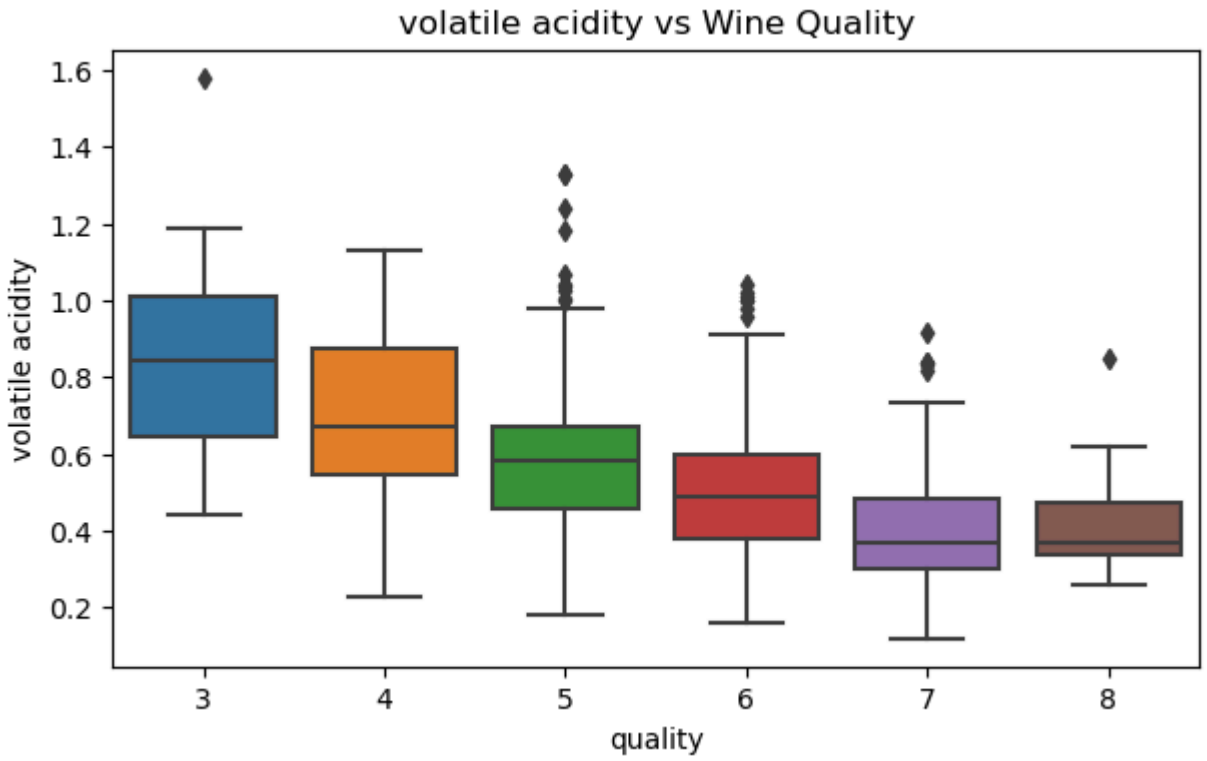
In [14]:
```
#Alcohol vs quality
```

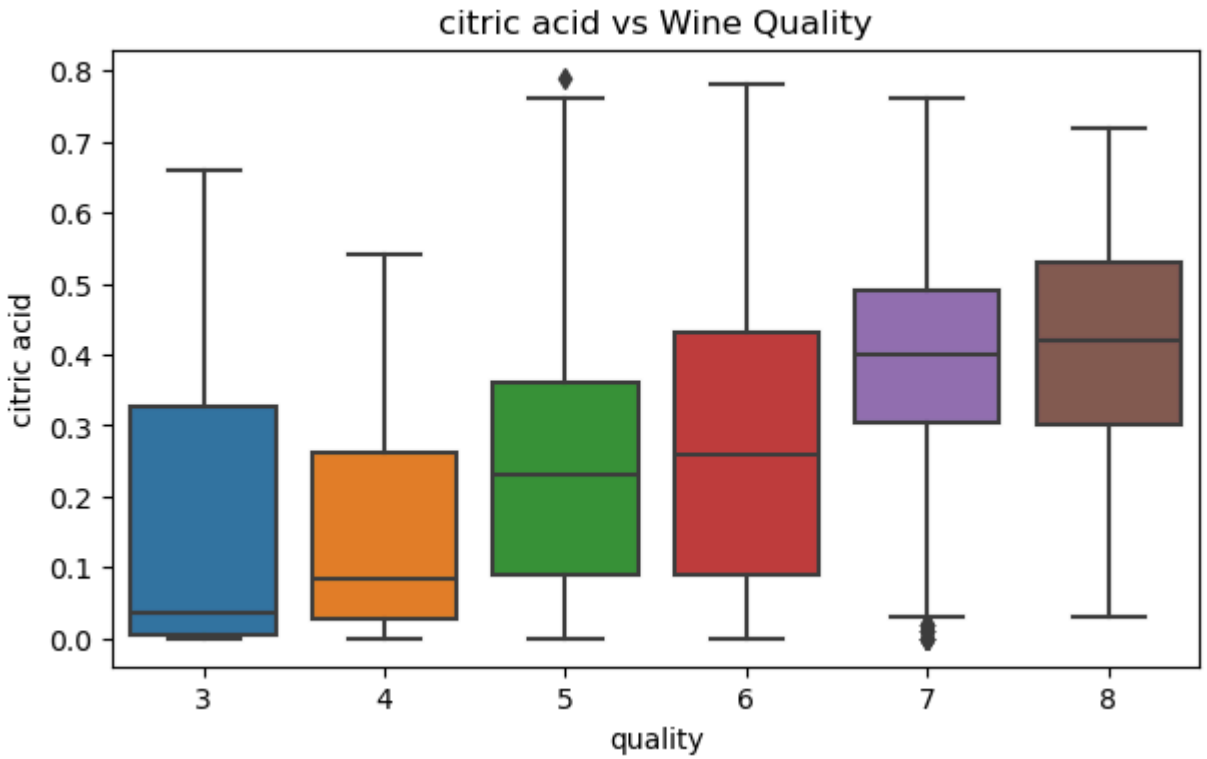
In [15]:
```
sns.boxplot(x='quality', y = 'alcohol', data=df)
plt.title('Alcohol vs Quality')
plt.show()
```

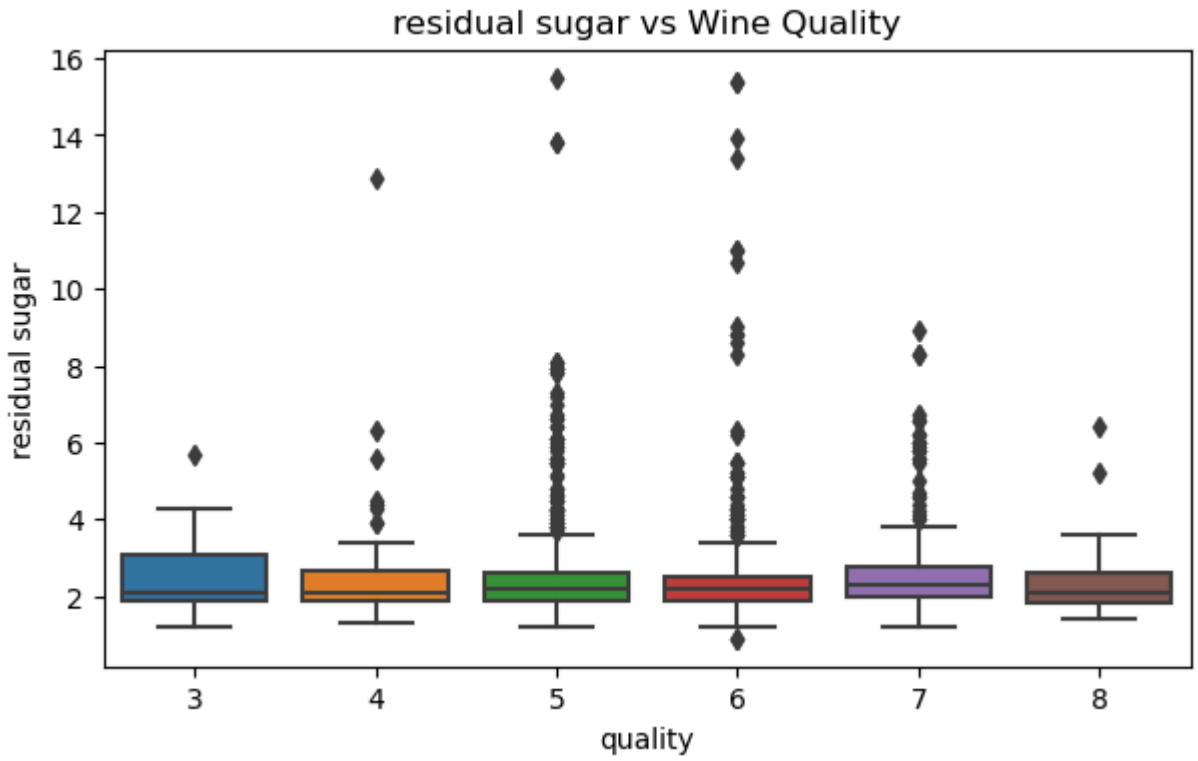


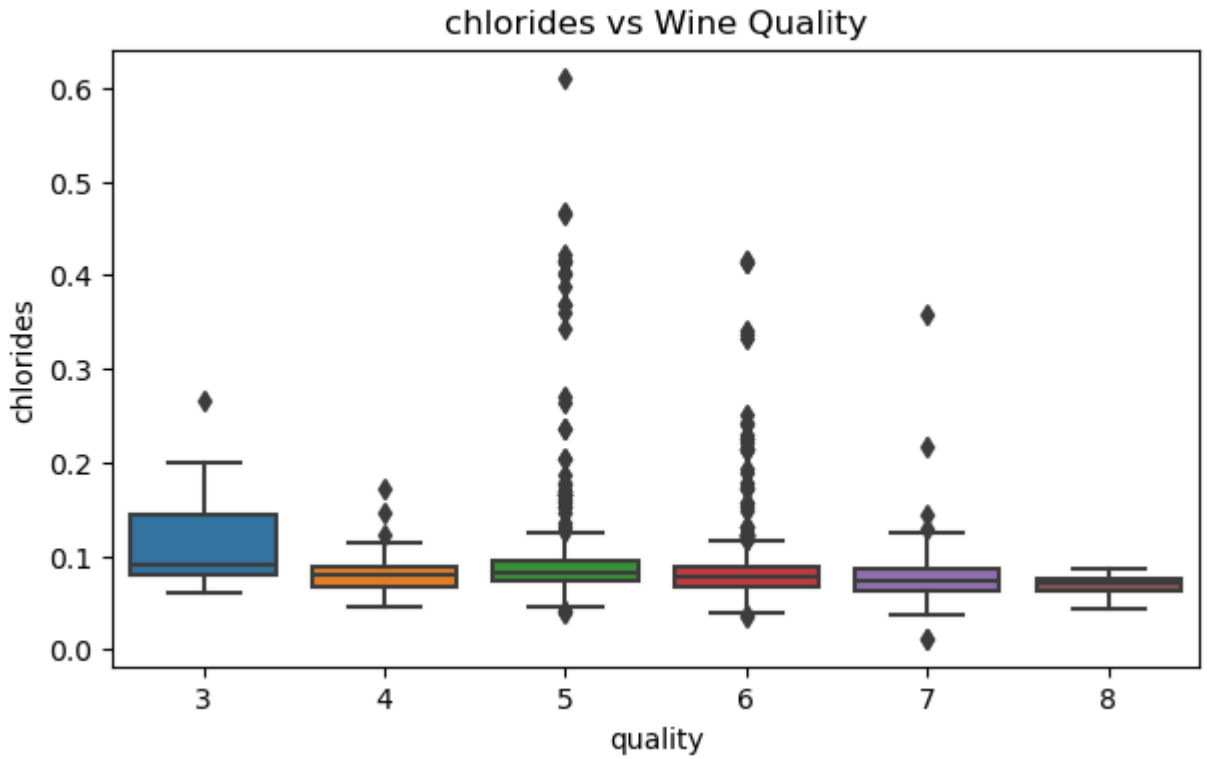
In [16]:
```
features = ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol']
for col in features:
    plt.figure(figsize=(7,4))
    sns.boxplot(x='quality', y =col,data=df)
    plt.title(f'{col} vs Wine Quality')
    plt.show()
```

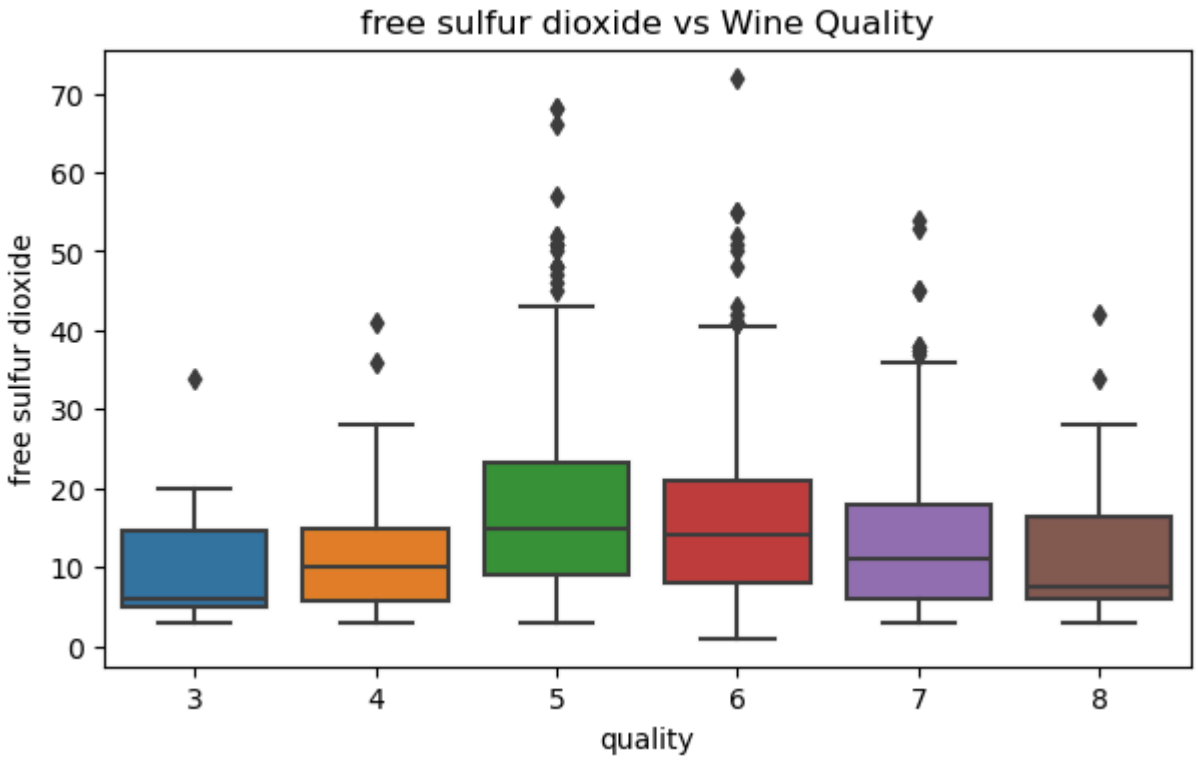

fixed acidity vs Wine Quality
fixed acidity
quality
16
14
12
10
8
6
3
4
5
6
7
8

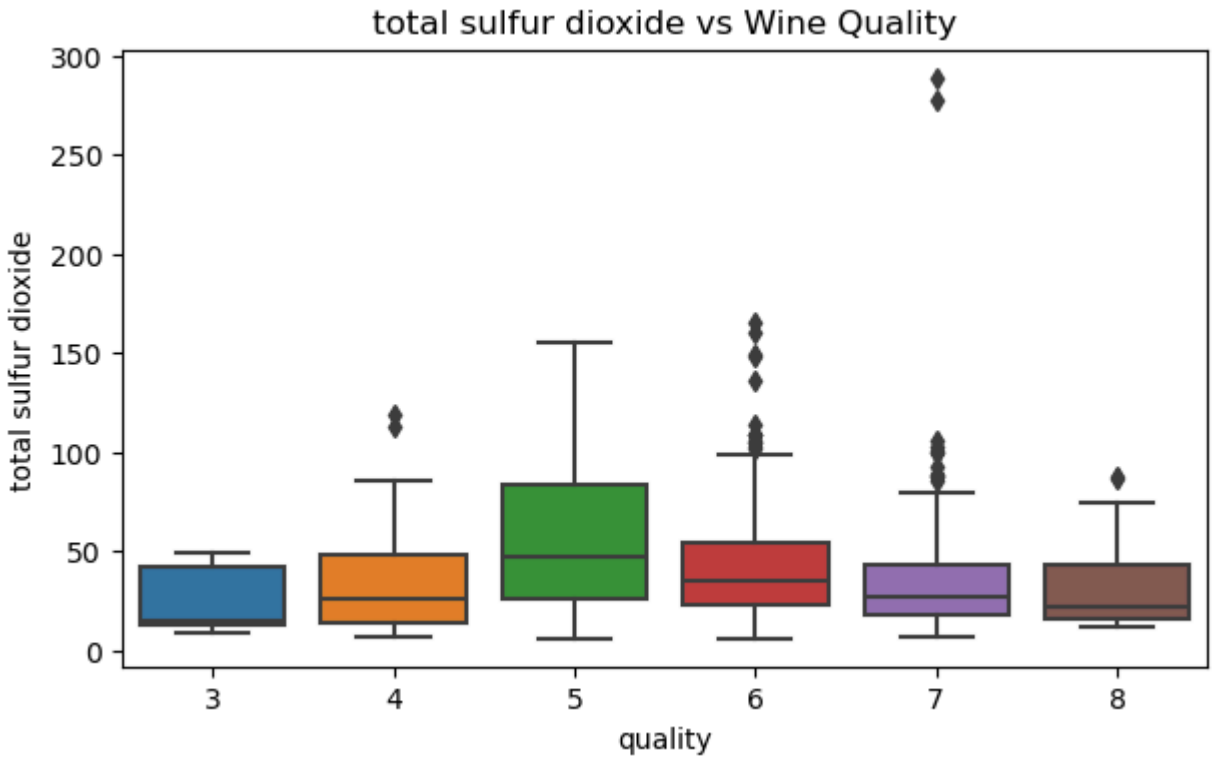

volatile acidity vs Wine Quality
volatile acidity
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
3
4
5
6
7
8
quality

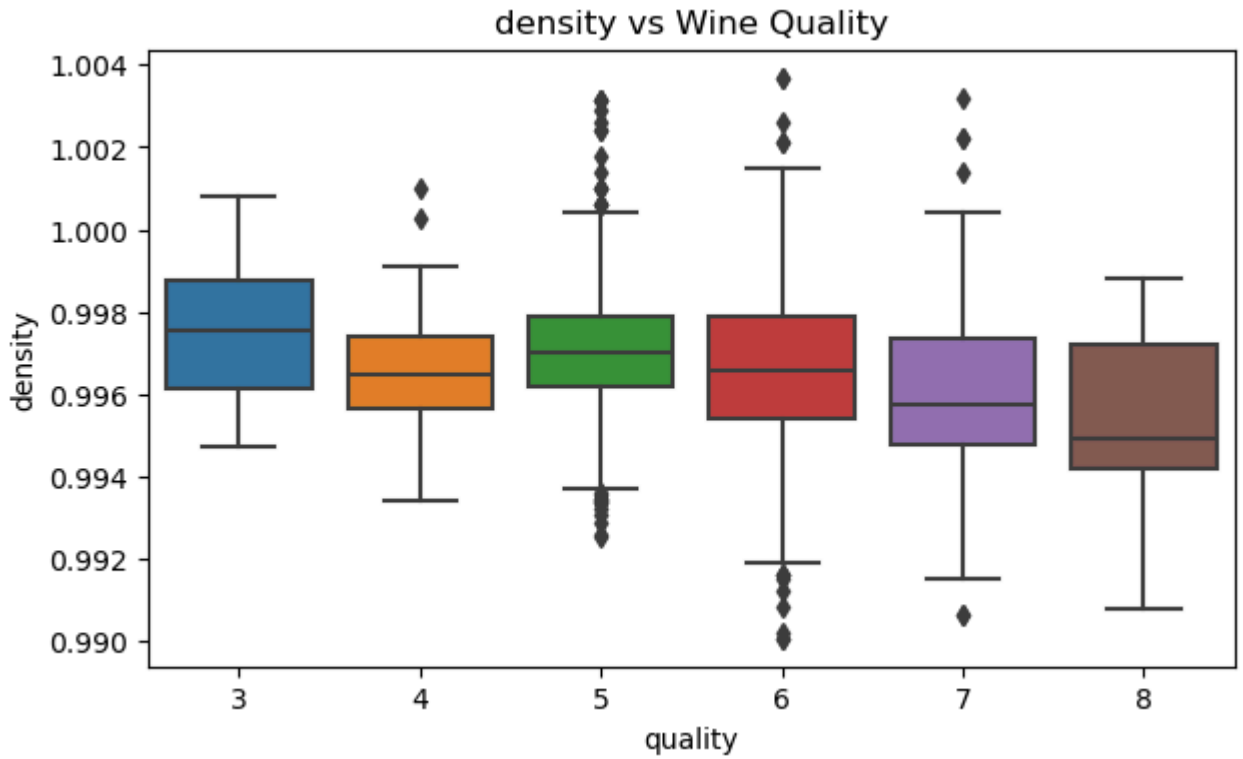

citric acid vs Wine Quality
citric acid
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
3
4
5
6
7
8
quality

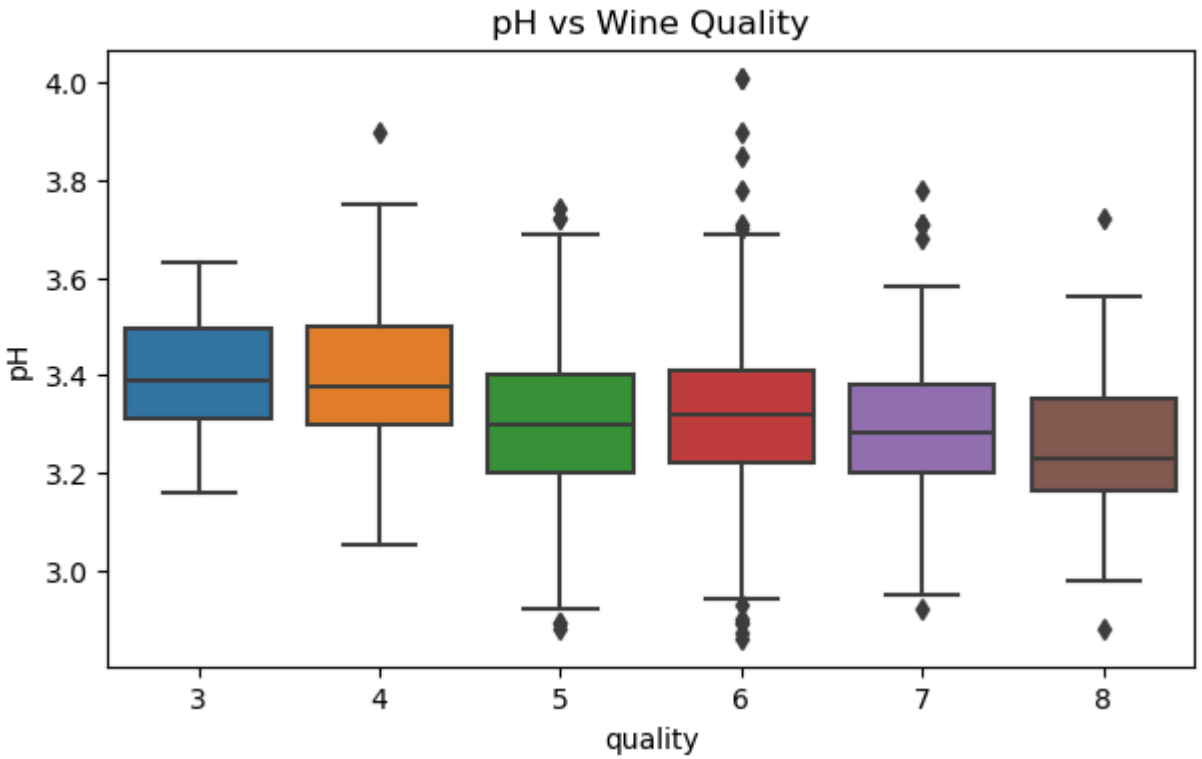

residual sugar vs Wine Quality
residual sugar
quality
16
14
12
10
8
6
4
2
3
4
5
6
7
8

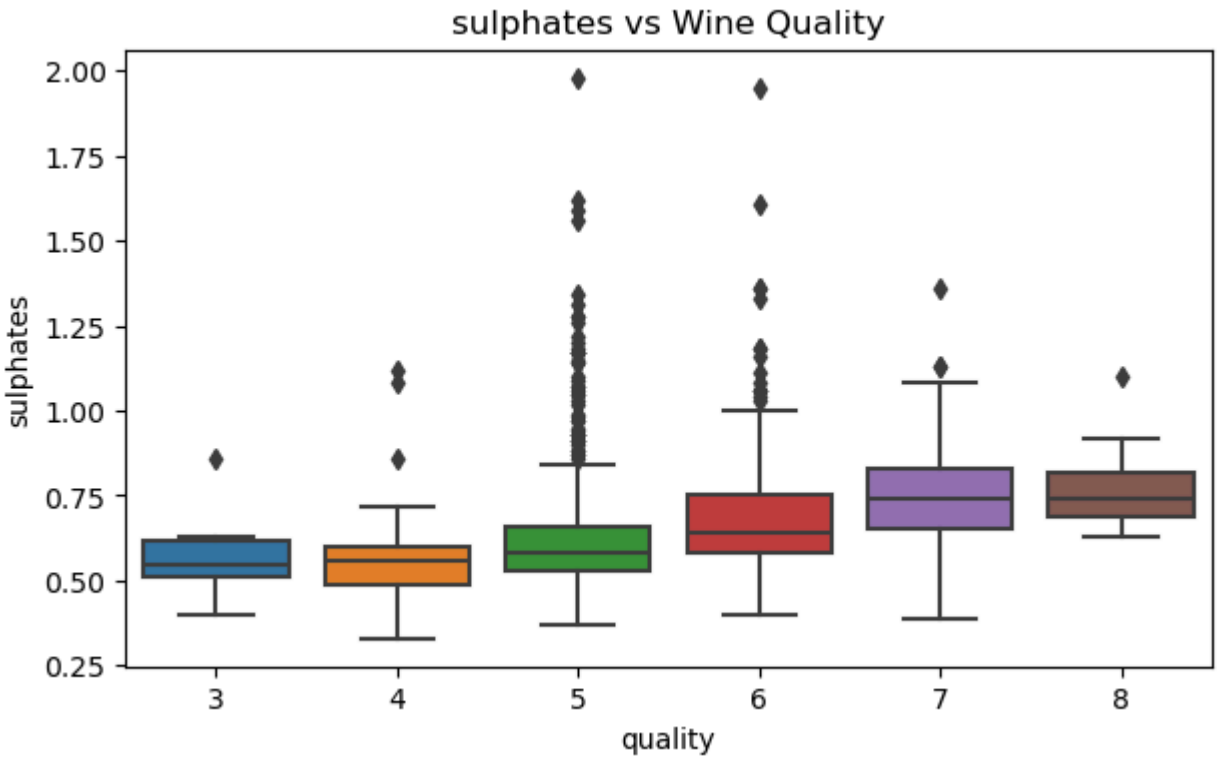

chlorides vs Wine Quality
chlorides
0.0
0.1
0.2
0.3
0.4
0.5
0.6
3
4
5
6
7
8
quality

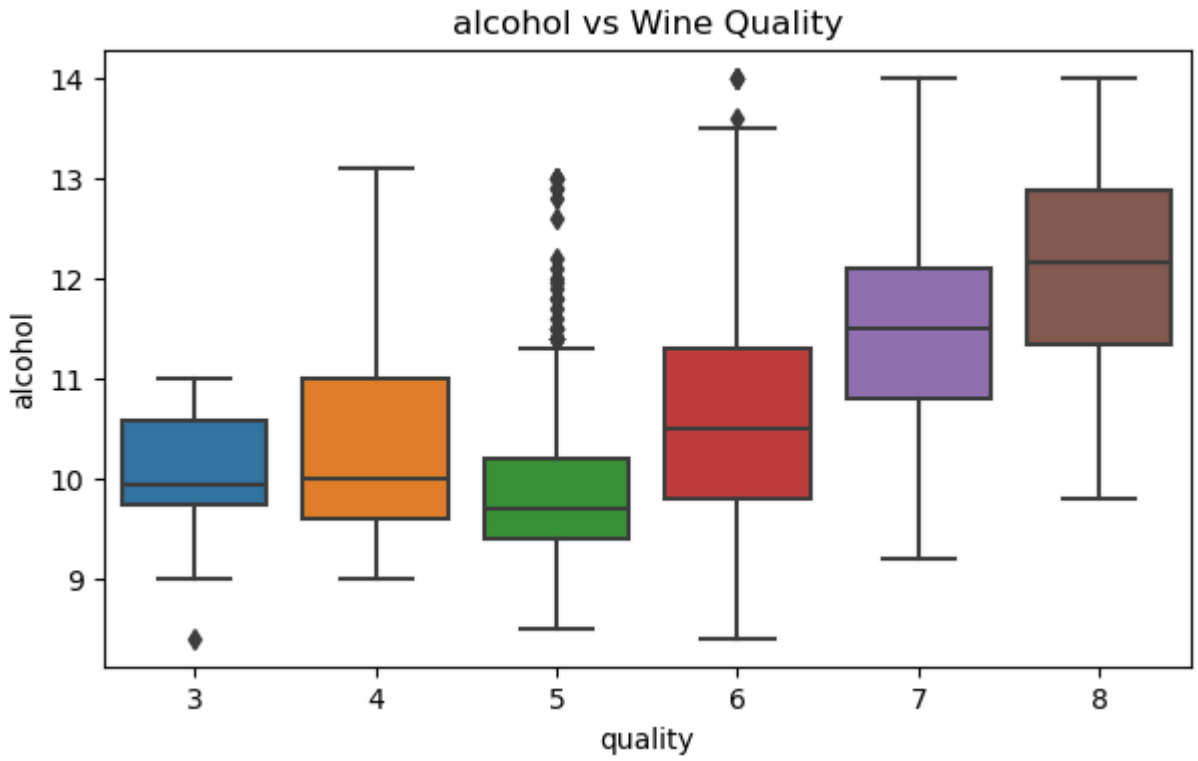

free sulfur dioxide vs Wine Quality
free sulfur dioxide
0
10
20
30
40
50
60
70
3
4
5
6
7
8
quality


total sulfur dioxide vs Wine Quality
total sulfur dioxide
300
250
200
150
100
50
0
3
4
5
6
7
8
quality

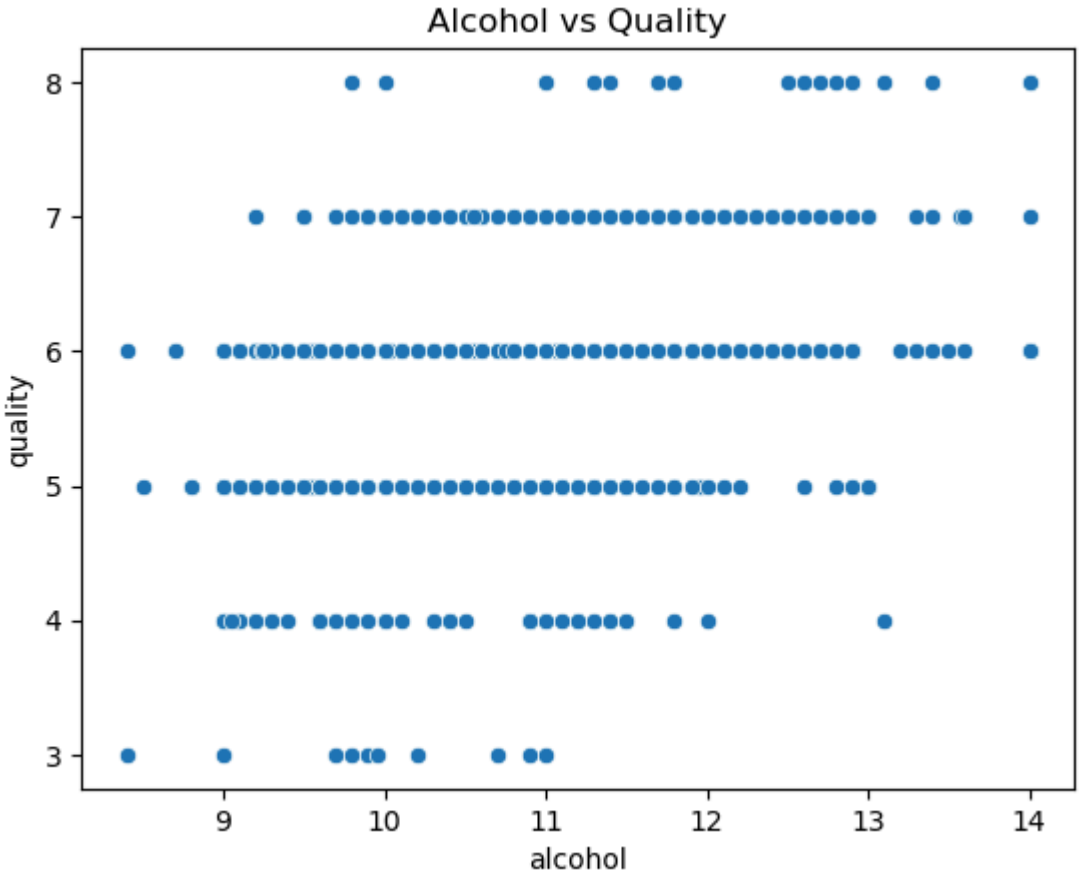

density vs Wine Quality
density
1.004
1.002
1.000
0.998
0.996
0.994
0.992
0.990
3
4
5
6
7
8
quality


pH vs Wine Quality
pH
4.0
3.8
3.6
3.4
3.2
3.0
3
4
5
6
7
8
quality

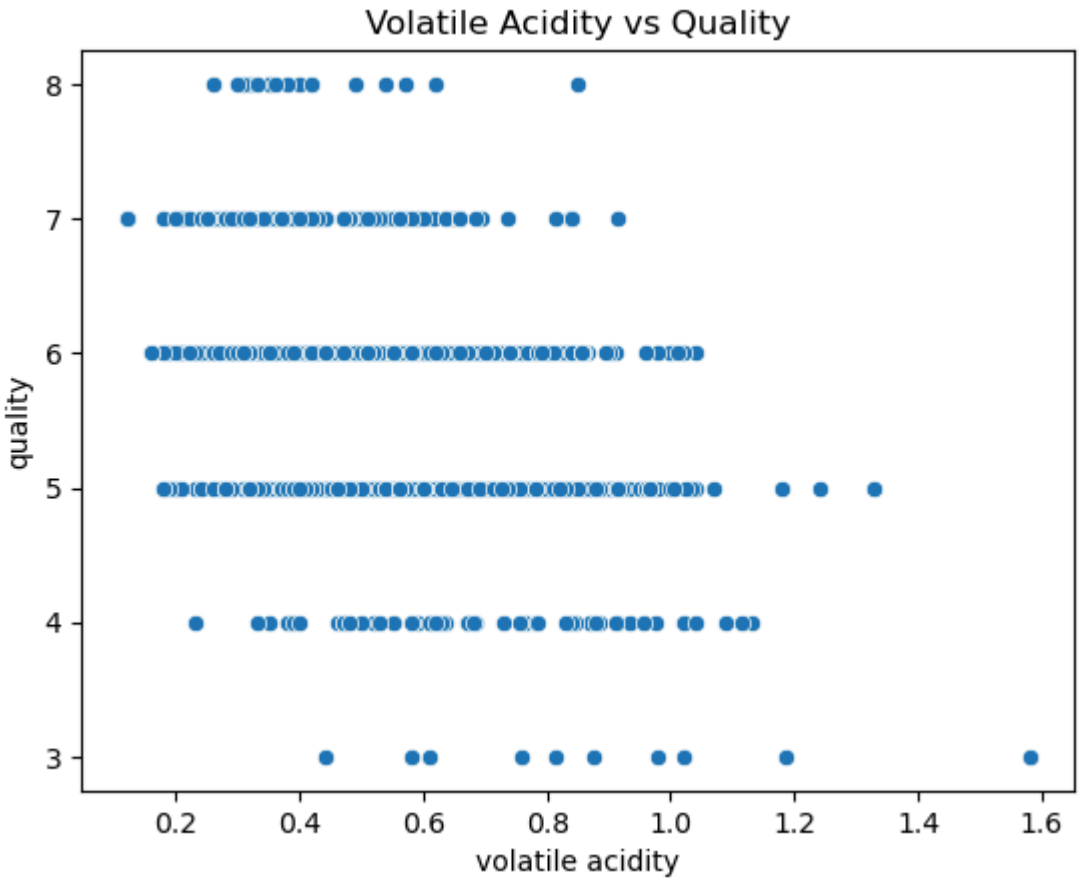

sulphates vs Wine Quality
sulphates
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
3
4
5
6
7
8
quality


alcohol vs Wine Quality
alcohol
quality
14
13
12
11
10
9
3
4
5
6
7
8

In [17]:
```
#scatter plot
sns.scatterplot(x='alcohol', y='quality', data =df)
plt.title('Alcohol vs Quality')
plt.show()
```



In [18]:
```
#scatter plot
sns.scatterplot(x='volatile acidity', y='quality', data =df)
plt.title('Volatile Acidity vs Quality')
plt.show()
```



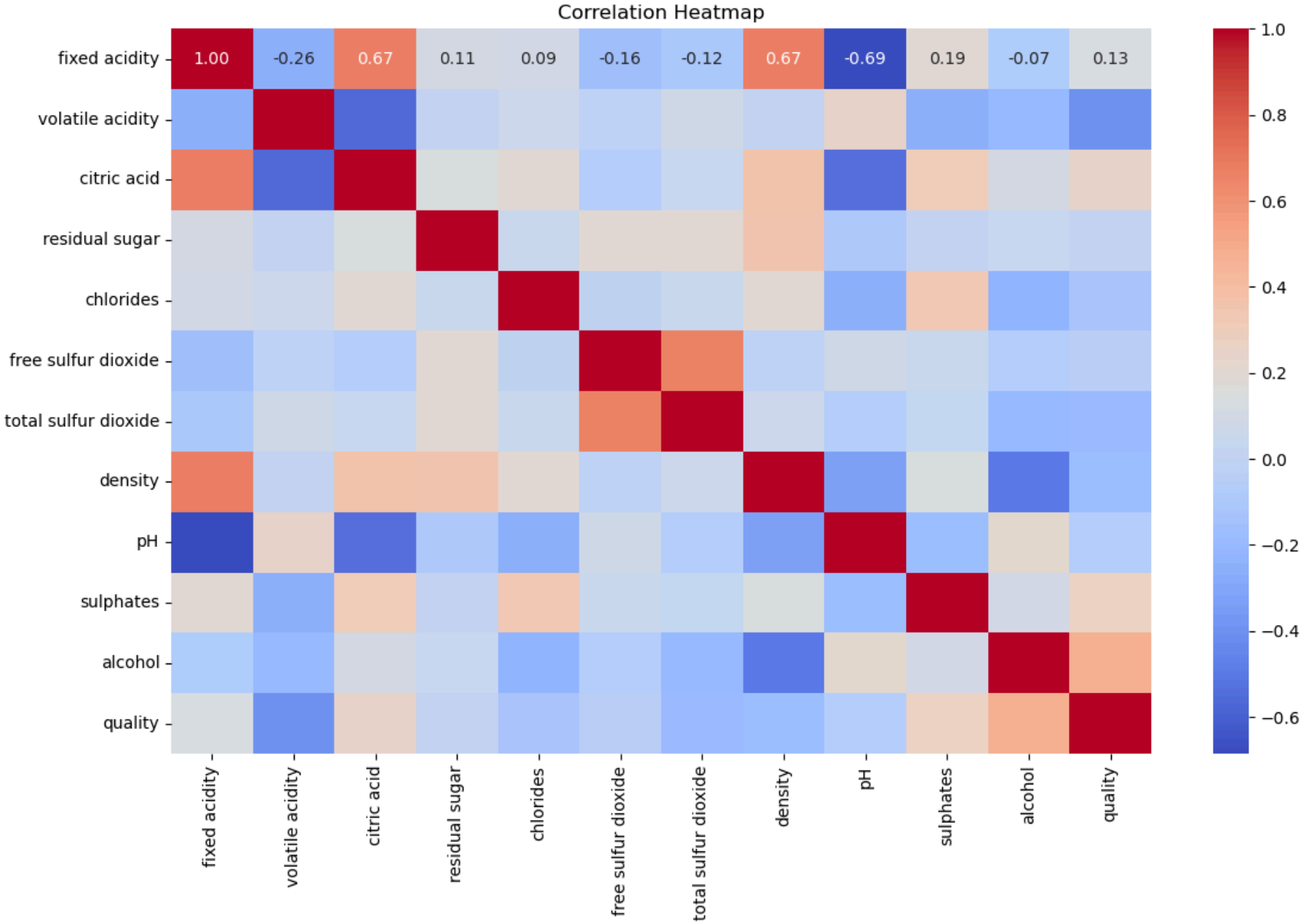
3 Multivariate Analysis

In [19]:
```
plt.figure(figsize =(12,8))

corr = df.corr()

sns.heatmap(corr,annot=True,cmap='coolwarm',fmt='.2f', annot_kws={'size':10})

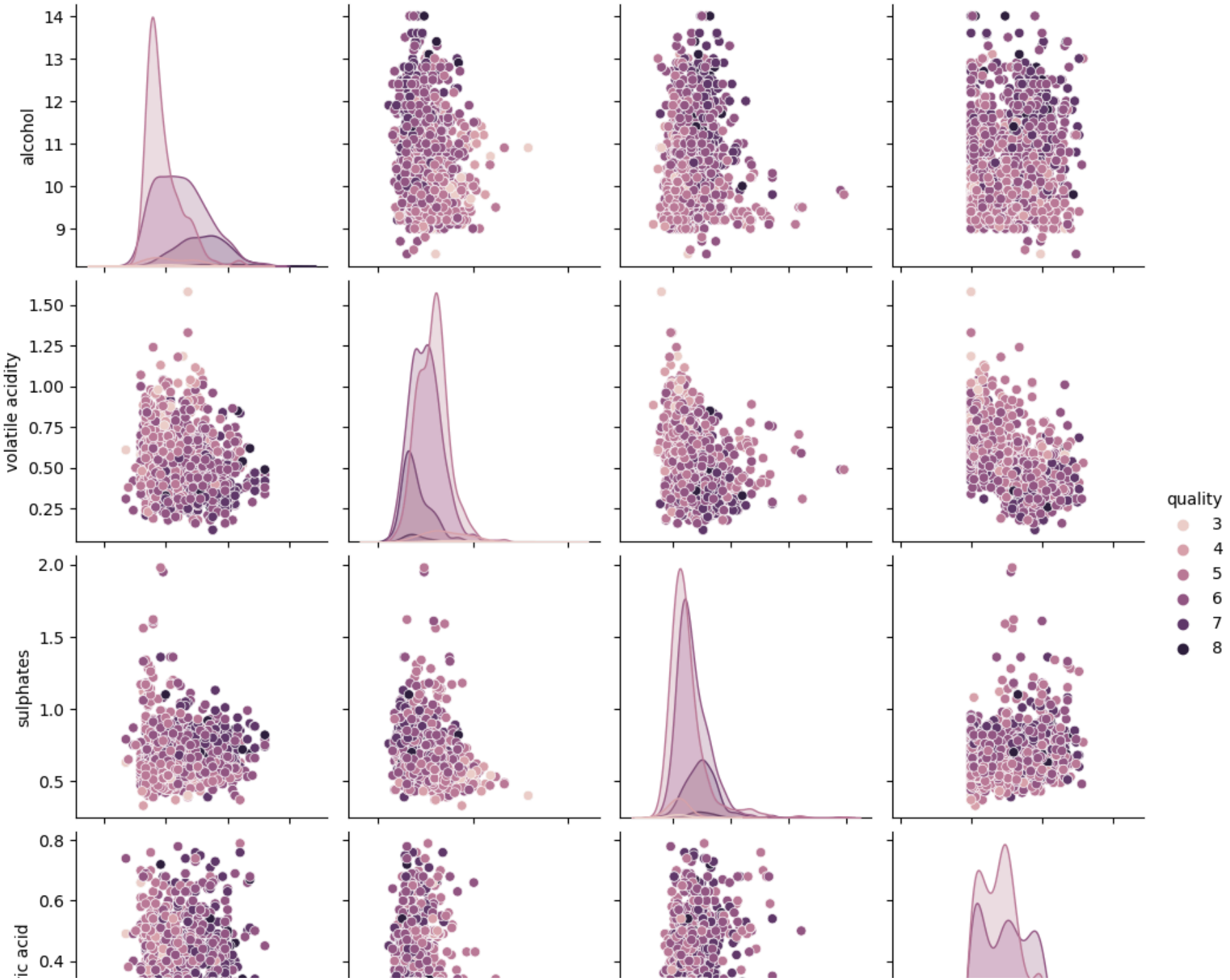
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

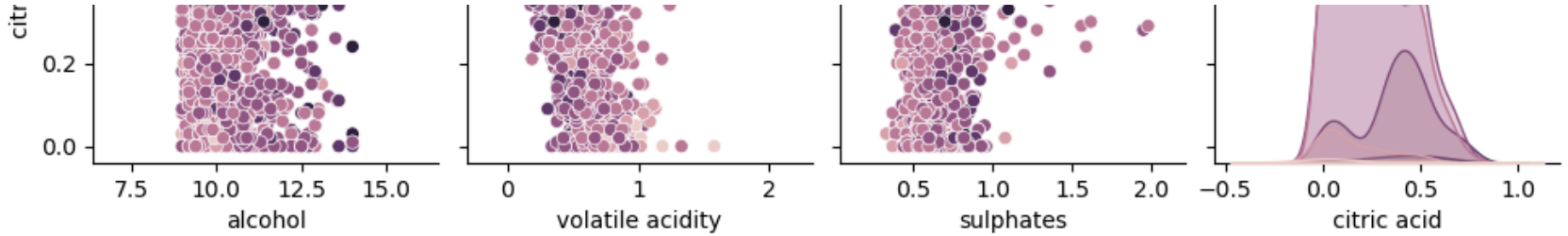

Correlation Heatmap
fixed acidity
volatile acidity
citric acid
residual sugar
chlorides
free sulfur dioxide
total sulfur dioxide
density
pH
sulphates
alcohol
quality
1.00 -0.26 0.67 0.11 0.09 -0.16 -0.12 0.67 -0.69 0.19 -0.07 0.13
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

In [20]:
```
#pair plot
selected =['alcohol','volatile acidity','sulphates','citric acid','quality']

sns.pairplot(df[selected], hue='quality')
plt.show()
```

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecate
d and will be removed in a future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```


alcohol
volatile acidity
sulphates
quality
3
4
5
6
7
8



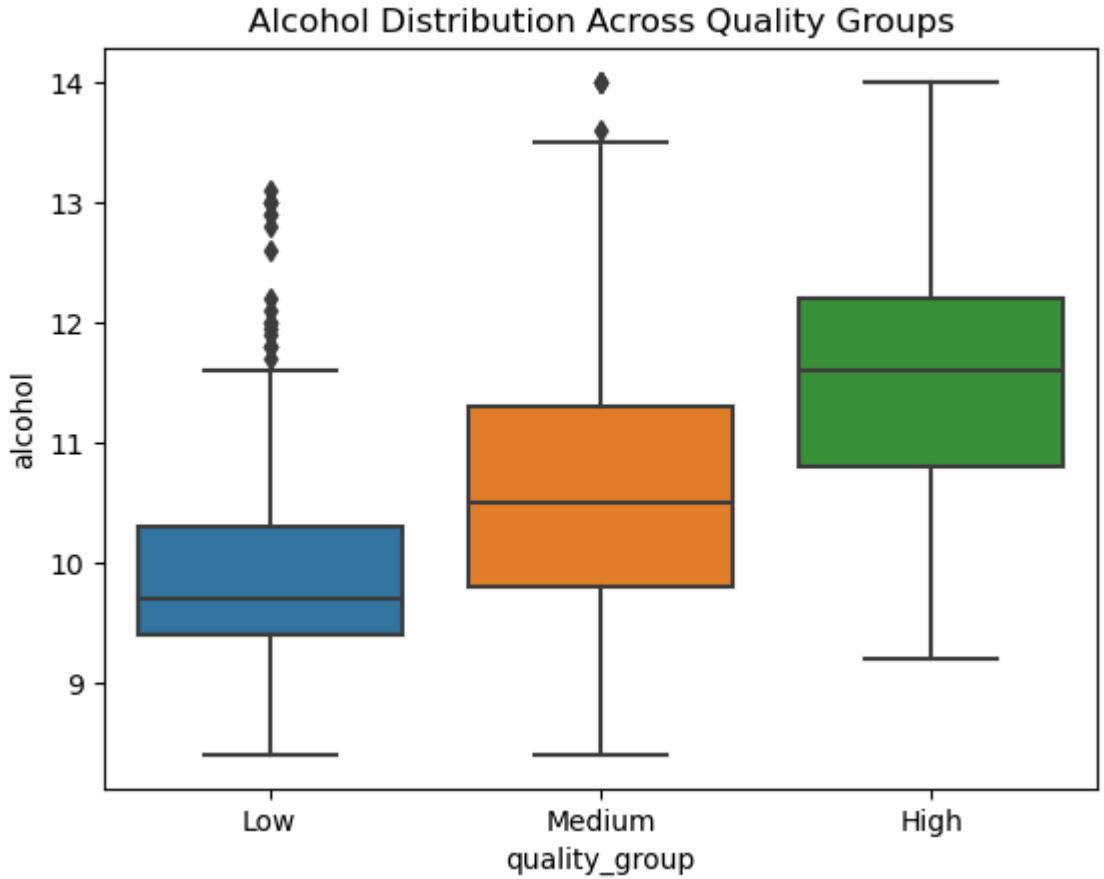
In [21]:
```
#multivariate analysis using quality groups

df['quality_group'] = pd.cut(
    df['quality'],
    bins = [0,5,6,10],
    labels=['Low','Medium','High']
)
```

In [22]:
```
sns.boxplot(
    x='quality_group',
    y='alcohol',
    data=df
)

plt.title("Alcohol Distribution Across Quality Groups")
plt.show()
```

```
C:\ProgramData\anaconda3\Lib\site-packages\seaborn\categorical.py:641: FutureWarning: The default of observed=False i
s deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavio
r or observed=True to adopt the future default and silence this warning.
  grouped_vals = vals.groupby(grouper)
```



In [23]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, classification_report,confusion_
matrix, ConfusionMatrixDisplay)
```

In [24]:
```
X =df.drop('quality', axis = 1)
y= df['quality']
```

In [25]:
```
print('X shape: ', X.shape)
print('y shape: ', y.shape)
```

```
X shape:  (1596, 12)
y shape:  (1596,)
```

In [26]:
```
print(y.value_counts().sort_index())
```

```
quality
3     10
4     52
5    680
6    637
7    199
8     18
Name: count, dtype: int64
```

In [27]:
```
X_train,X_test,y_train, y_test = train_test_split(X,y,test_size = 0.30, random_state =57, stratify =y)
```

In [28]:
```
criterion = 'entropy'
```

In [29]:
```
dt_model = DecisionTreeClassifier(criterion='entropy', random_state= 57)
```

In [30]:
```
#multivariate analysis using quality groups

df['quality_group'] = pd.cut(
    df['quality'],
    bins = [0,5,6,10],
    labels=['Low','Medium','High']
)
```

In [31]:
```
print(X.columns)
```

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol', 'quality_group'],
      dtype='object')
```

In [32]:
```
print(X.select_dtypes(exclude='number').columns)
```

```
Index(['quality_group'], dtype='object')
```

In [33]:
```
y=df['quality']
```

In [34]:
```
X=df.drop(columns=['quality','quality_group'],errors='ignore')
```

In [35]:
```
print(X.columns)
print(X.dtypes)
```

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
       'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
       'pH', 'sulphates', 'alcohol'],
      dtype='object')
fixed acidity           float64
volatile acidity        float64
citric acid             float64
residual sugar          float64
chlorides               float64
free sulfur dioxide     float64
total sulfur dioxide    float64
density                 float64
pH                      float64
sulphates               float64
alcohol                 float64
dtype: object
```

In [36]:
```
X_train,X_test,y_train, y_test = train_test_split(X,y,test_size = 0.30, random_state =57, stratify =y)
```

In [37]:
```
print('X_train : ', X_train.shape)
print('X_test: ', X_test.shape)
print('y_train: ', y_train.shape)
print('y_test: ', y_test.shape)
```

```
X_train :  (1117, 11)
X_test:  (479, 11)
y_train:  (1117,)
y_test:  (479,)
```

In [38]:
```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(criterion= 'entropy',random_state =57)
```

In [39]:
```
dt_model.fit(X_train,y_train)
```

Out[39]:
```
DecisionTreeClassifier(criterion='entropy', random_state=57)
```

In [40]:
```
y_train_pred = dt_model.predict(X_train)
y_test_pred = dt_model.predict(X_test)
```

In [41]:
```
train_accuracy = accuracy_score(y_train, y_train_pred)
print("Training Accuracy", train_accuracy)#overfitting
```

```
Training Accuracy 1.0
```

In [42]:
```
test_accuracy = accuracy_score(y_test, y_test_pred)
print("Testing Accuracy", test_accuracy) #overfitting
```

```
Testing Accuracy 0.6304801670146137
```

In [43]:
```
print("Precision: ",
      precision_score(y_test, y_test_pred, average ='weighted'))
print("Recall: ",
      recall_score(y_test, y_test_pred, average ='weighted'))
print("F1 Score: ",
      f1_score(y_test, y_test_pred, average ='weighted'))
```

```
Precision:  0.6084294080928228
Recall:  0.6304801670146137
F1 Score:  0.6177909809326186

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: UndefinedMetricWarning: Precision
is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control thi
s behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

In [44]:
```
print(classification_report(y_test,y_test_pred))
```

```
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         3
           4       0.00      0.00      0.00        16
           5       0.70      0.76      0.73       204
           6       0.60      0.63      0.62       191
           7       0.57      0.45      0.50        60
           8       0.00      0.00      0.00         5

    accuracy                           0.63       479
   macro avg       0.31      0.31      0.31       479
weighted avg       0.61      0.63      0.62       479


C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: UndefinedMetricWarning: Precision
and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter t
o control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: UndefinedMetricWarning: Precision
and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter t
o control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: UndefinedMetricWarning: Precision
and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter t
o control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

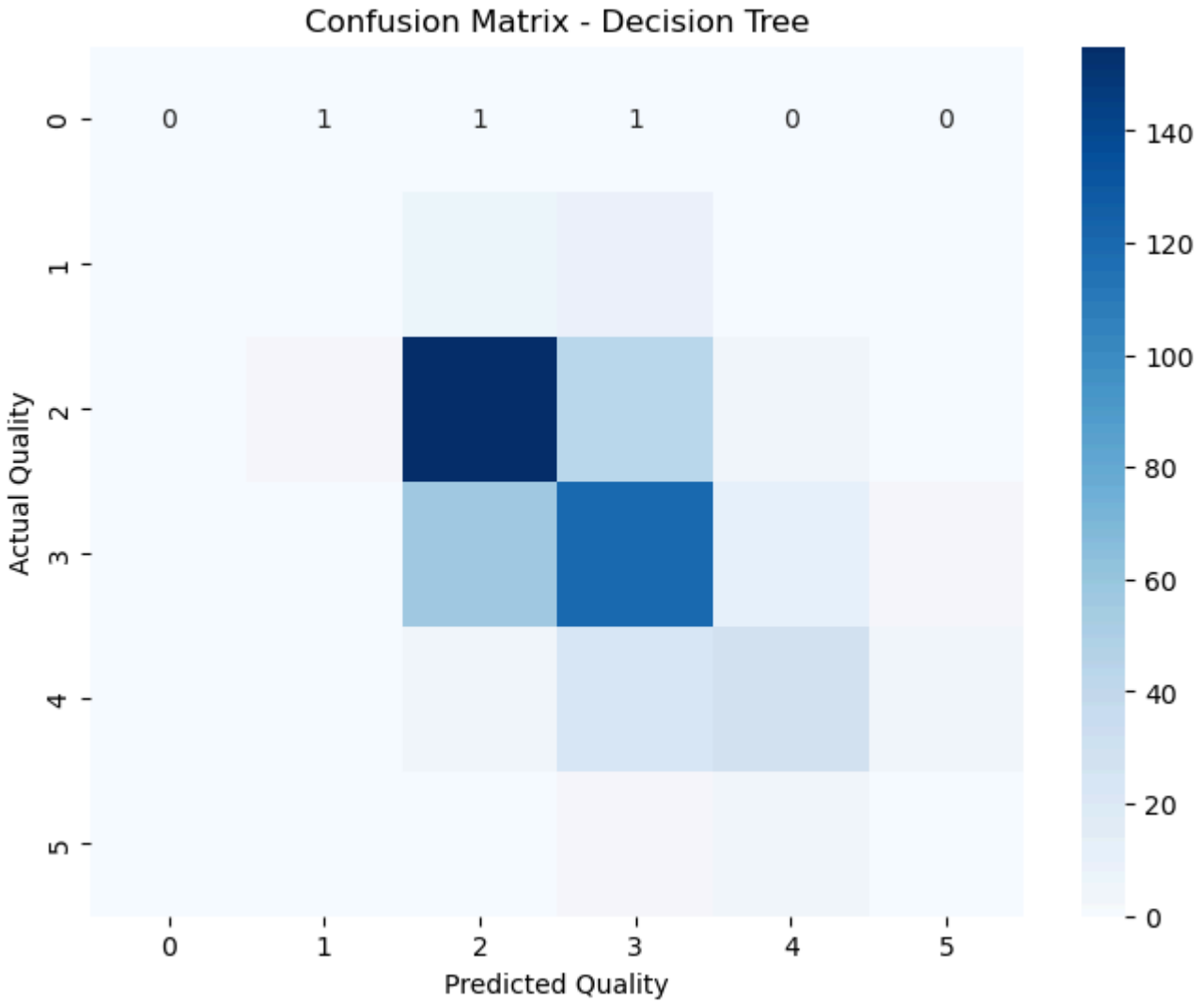
In [45]:
```
#Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)

plt.figure(figsize=(8,6))

sns.heatmap(cm, annot =True, fmt='d', cmap='Blues')

plt.xlabel('Predicted Quality')
plt.ylabel('Actual Quality')
plt.title('Confusion Matrix - Decision Tree')

plt.show()
```



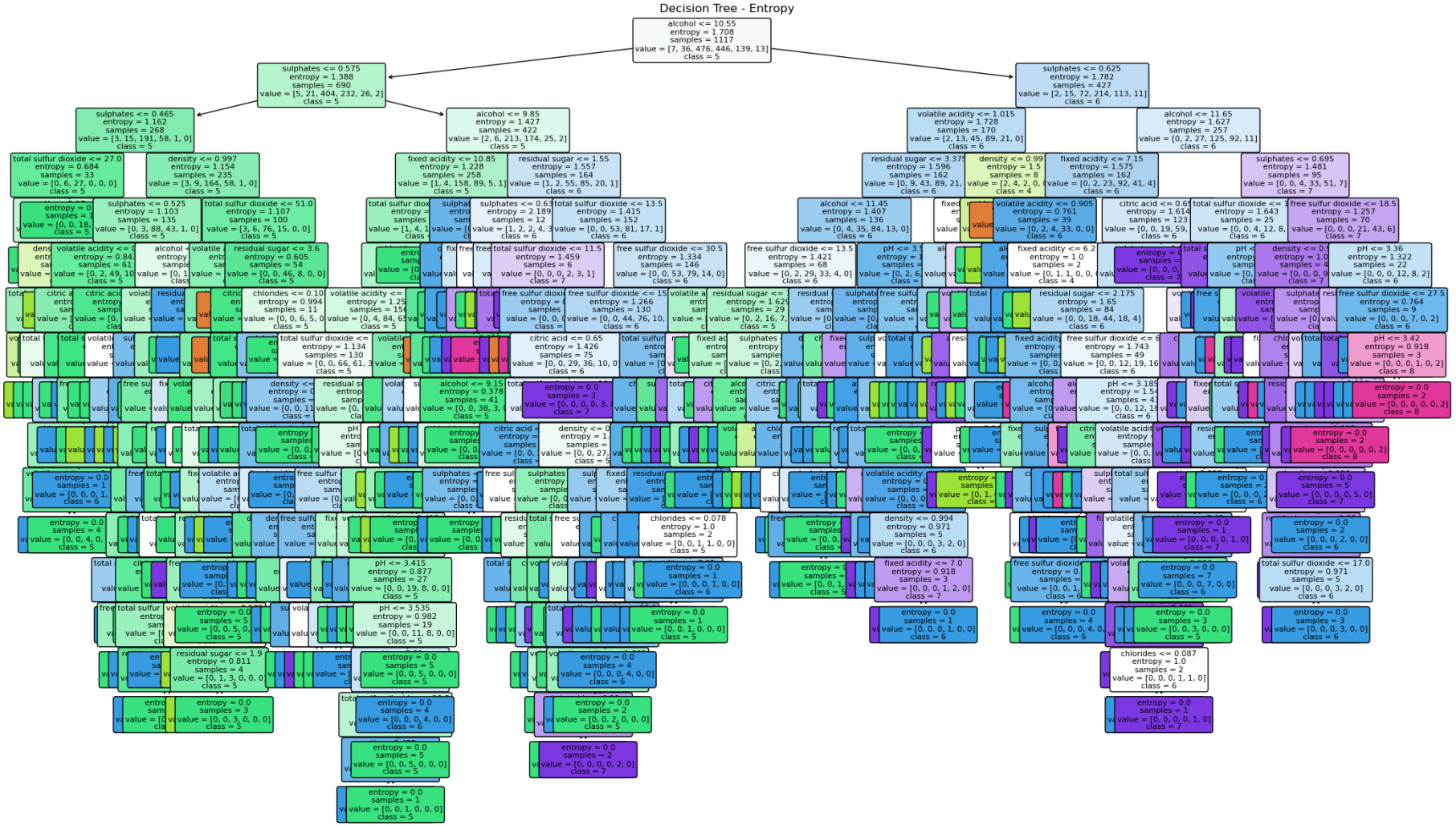
In [46]:
```
print('Tree Depth', dt_model.get_depth())
print('Number Of Leaves: ', dt_model.get_n_leaves())
```

```
Tree Depth 17
Number Of Leaves:  279
```

In [47]:
```
plt.figure(figsize=(25, 15))

plot_tree(
    dt_model,
    feature_names = X.columns,
    class_names=[str(c) for c in sorted(y.unique())],
    filled =True,
    rounded =True,
    fontsize =8
)

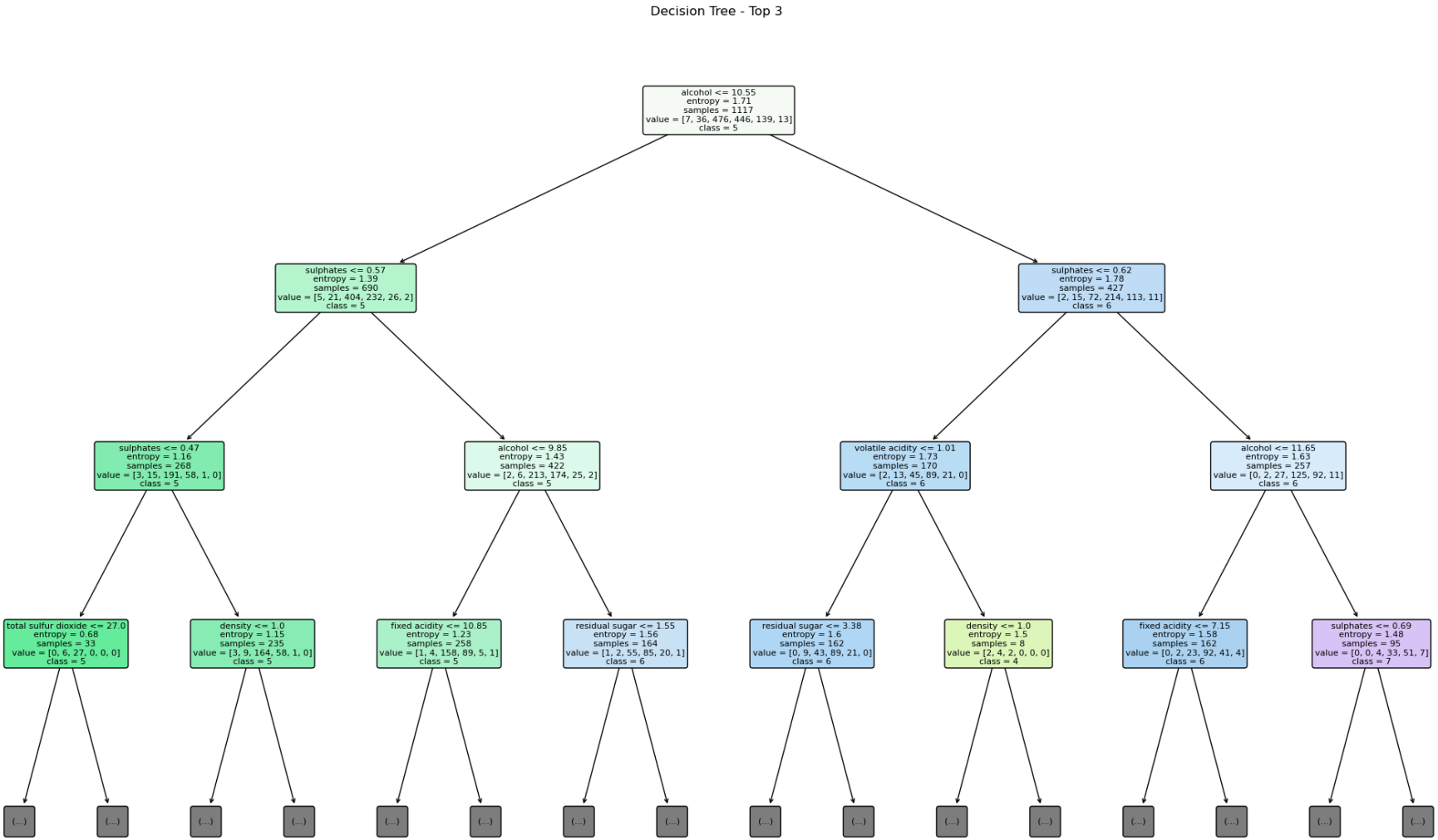
plt.title("Decision Tree - Entropy")
plt.show()
```


Decision Tree - Entropy
alcohol <= 10.55
entropy = 1.708
samples = 1117
value = [7, 36, 476, 446, 139, 13]
class = 5
sulphates <= 0.575
entropy = 1.388
samples = 690
value = [5, 21, 404, 232, 26, 2]
class = 5
sulphates <= 0.625
entropy = 1.782
samples = 427
value = [2, 15, 72, 214, 113, 11]
class = 6
sulphates <= 0.465
entropy = 1.162
samples = 268
value = [3, 15, 191, 58, 1, 0]
class = 5
alcohol <= 9.85
entropy = 1.427
samples = 422
value = [2, 6, 213, 174, 25, 2]
class = 5
volatile acidity <= 1.015
entropy = 1.728
samples = 170
value = [2, 13, 45, 89, 21, 0]
class = 6
alcohol <= 11.65
entropy = 1.627
samples = 257
value = [0, 2, 27, 125, 92, 11]
class = 6
total sulfur dioxide <= 27.0
entropy = 0.684
samples = 33
value = [0, 6, 27, 0, 0, 0]
class = 5
density <= 0.997
entropy = 1.154
samples = 235
value = [3, 9, 164, 58, 1, 0]
class = 5
fixed acidity <= 10.85
entropy = 1.228
samples = 258
value = [1, 4, 158, 89, 5, 1]
class = 5
residual sugar <= 1.55
entropy = 1.557
samples = 164
value = [1, 2, 55, 85, 20, 1]
class = 6
residual sugar <= 3.375
entropy = 1.596
samples = 162
value = [0, 9, 43, 89, 21, 0]
class = 6
density <= 0.99
entropy = 1.5
samples = 8
value = [2, 4, 2, 0, 0, 0]
class = 4
fixed acidity <= 7.15
entropy = 1.575
samples = 162
value = [0, 2, 23, 92, 41, 4]
class = 6
sulphates <= 0.695
entropy = 1.481
samples = 95
value = [0, 0, 4, 33, 51, 7]
class = 7
entropy = 0.0
samples = 1
value = [0, 0, 18, 0, 0, 0]
class = 5
sulphates <= 0.525
entropy = 1.103
samples = 135
value = [0, 3, 88, 43, 1, 0]
class = 5
total sulfur dioxide <= 51.0
entropy = 1.107
samples = 100
value = [3, 6, 76, 15, 0, 0]
class = 5
total sulfur dioxide <= 13.5
entropy = 1.415
samples = 152
value = [0, 0, 53, 81, 17, 1]
class = 6
volatile acidity <= 0.905
entropy = 0.761
samples = 39
value = [0, 2, 4, 33, 0, 0]
class = 6
citric acid <= 0.63
entropy = 1.614
samples = 123
value = [0, 0, 19, 59, 21, 4]
class = 6
free sulfur dioxide <= 18.5
entropy = 1.257
samples = 70
value = [0, 0, 0, 21, 43, 6]
class = 7
density <= 0.995
entropy = 0.605
samples = 54
value = [0, 0, 46, 8, 0, 0]
class = 5
free sulfur dioxide <= 30.5
entropy = 1.334
samples = 146
value = [0, 0, 53, 79, 14, 0]
class = 6
free sulfur dioxide <= 11.5
entropy = 1.459
samples = 6
value = [0, 0, 0, 2, 3, 1]
class = 7
free sulfur dioxide <= 13.5
entropy = 1.421
samples = 68
value = [0, 2, 29, 33, 4, 0]
class = 6
fixed acidity <= 6.2
entropy = 1.0
samples = 2
value = [0, 1, 1, 0, 0, 0]
class = 4
pH <= 3.36
entropy = 1.322
samples = 22
value = [0, 0, 0, 12, 8, 2]
class = 6
residual sugar <= 2.175
entropy = 1.65
samples = 84
value = [0, 0, 18, 44, 18, 4]
class = 6
free sulfur dioxide <= 27.5
entropy = 0.764
samples = 9
value = [0, 0, 0, 7, 0, 2]
class = 6
citric acid <= 0.65
entropy = 1.426
samples = 75
value = [0, 0, 29, 36, 10, 0]
class = 6
pH <= 3.42
entropy = 0.918
samples = 3
value = [0, 0, 0, 1, 0, 2]
class = 8
alcohol <= 9.15
entropy = 0.378
samples = 41
value = [0, 0, 38, 3, 0, 0]
class = 5
density <= 0.994
entropy = 0.971
samples = 5
value = [0, 0, 0, 3, 2, 0]
class = 6
pH <= 3.415
entropy = 0.877
samples = 27
value = [0, 0, 19, 8, 0, 0]
class = 5
chlorides <= 0.078
entropy = 1.0
samples = 2
value = [0, 0, 1, 1, 0, 0]
class = 5
fixed acidity <= 7.0
entropy = 0.918
samples = 3
value = [0, 0, 0, 1, 2, 0]
class = 7
pH <= 3.535
entropy = 0.982
samples = 19
value = [0, 0, 11, 8, 0, 0]
class = 5
residual sugar <= 1.9
entropy = 0.811
samples = 4
value = [0, 1, 3, 0, 0, 0]
class = 5
chlorides <= 0.087
entropy = 1.0
samples = 2
value = [0, 0, 0, 1, 1, 0]
class = 6

In [48]:
```
plt.figure(figsize=(25, 15))

plot_tree(
    dt_model,
    feature_names = X.columns,
    class_names=[str(c) for c in sorted(y.unique())],
    filled =True,
    rounded =True,
    fontsize =8,
    impurity = True,
    precision=2,
    max_depth=3
)

plt.title("Decision Tree - Top 3 ")
plt.show()
```


Decision Tree - Top 3
alcohol <= 10.55
entropy = 1.71
samples = 1117
value = [7, 36, 476, 446, 139, 13]
class = 5
sulphates <= 0.57
entropy = 1.39
samples = 690
value = [5, 21, 404, 232, 26, 2]
class = 5
sulphates <= 0.62
entropy = 1.78
samples = 427
value = [2, 15, 72, 214, 113, 11]
class = 6
sulphates <= 0.47
entropy = 1.16
samples = 268
value = [3, 15, 191, 58, 1, 0]
class = 5
alcohol <= 9.85
entropy = 1.43
samples = 422
value = [2, 6, 213, 174, 25, 2]
class = 5
volatile acidity <= 1.01
entropy = 1.73
samples = 170
value = [2, 13, 45, 89, 21, 0]
class = 6
alcohol <= 11.65
entropy = 1.63
samples = 257
value = [0, 2, 27, 125, 92, 11]
class = 6
total sulfur dioxide <= 27.0
entropy = 0.68
samples = 33
value = [0, 6, 27, 0, 0, 0]
class = 5
density <= 1.0
entropy = 1.15
samples = 235
value = [3, 9, 164, 58, 1, 0]
class = 5
fixed acidity <= 10.85
entropy = 1.23
samples = 258
value = [1, 4, 158, 89, 5, 1]
class = 5
residual sugar <= 1.55
entropy = 1.56
samples = 164
value = [1, 2, 55, 85, 20, 1]
class = 6
residual sugar <= 3.38
entropy = 1.6
samples = 162
value = [0, 9, 43, 89, 21, 0]
class = 6
density <= 1.0
entropy = 1.5
samples = 8
value = [2, 4, 2, 0, 0, 0]
class = 4
fixed acidity <= 7.15
entropy = 1.58
samples = 162
value = [0, 2, 23, 92, 41, 4]
class = 6
sulphates <= 0.69
entropy = 1.48
samples = 95
value = [0, 0, 4, 33, 51, 7]
class = 7
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)
(...)

HyperParameter Tuning

In [ ]:
```
from sklearn.model_selection import GridSearchCV
import warnings
warnings.filterwarnings('ignore')

param_grid ={
    'criteria' : ['entropy'],
    'max_depth':[3,5,7,10,None],
    'min_samples_split':[2,5,]
```